NATIONAL DIPLOMA IN INFORMATION TECHNOLOGY

SYSTEMS DEVELOPMENT PROJECT DOCUMENTATION

LADO

Learn As you Do

A browser-based learn-by-doing platform with live 3D visualisation for practising C++, C# and SQL

Submitted by

Ireen Jakazira

Institution: Masvingo Polytechnic, Masvingo

Supervisor: Mrs Chivingira

In partial fulfilment of the requirements of the National Diploma in Information Technology (Software Development)

Masvingo, 2026

ACKNOWLEDGEMENTS

I thank Almighty God for the strength, health and wisdom that carried me through this project.

My sincere gratitude goes to my supervisor, Mrs Chivingira, for her guidance, patience and honest feedback at every stage, from the first proposal to the final document. Her insistence on clear objectives and careful testing shaped this project.

I thank the lecturers and staff of the Information Technology department of Masvingo Polytechnic for their time during the interviews and for sharing the problems students face when they practise programming and databases. I also thank the students who tried the prototype and gave honest comments, and my classmates for their encouragement and for being the first to say “show me what the code does”.

Finally, I thank my family and friends for their love, support and understanding during the many late nights it took to finish this work.

DEDICATION

This project is dedicated to my family, whose love and sacrifice made my studies possible, and to my supervisor, Mrs Chivingira, who believed that a student could build something useful.

It is also dedicated to every learner who has ever felt stuck in front of a blinking cursor and kept trying. May this work help you to see that code is not magic: it is something you can watch, understand and master.

TABLE OF CONTENTS

Acknowledgements....i
Dedication....ii
Table of Contents....iii
List of Acronyms....v
List of Tables....vi
List of Figures....viii
List of Appendices....ix
Chapter 1: Project Identification....1
1.0 Introduction....1
1.1 Concerned Organisation....1
1.2 Current System Overview....3
Chapter 2: Feasibility Study....8
2.1 Introduction....8
2.2 Current System Detail....8
2.3 Description of System Alternatives....11
2.4 Feasibility of Each Alternative....13
2.5 Research Methodology....15
Chapter 3: Analysis Phase (For the New System)....16
3.1 Introduction....16
3.2 New System Details....16
3.3 Research Methodologies....19
3.4 Requirements Analysis....19
3.5 Conclusion....21
Chapter 4: Design Phase....22
4.1 Introduction....22
4.2 System Design....22
4.3 Architectural Design....28
4.4 Physical Design....28
4.5 Design of Inputs, Processes, Outputs, Controls, Files and Interface....29
4.6 Conclusion....32
Chapter 5: Implementation Phase....33
5.1 Introduction....33
5.2 Implementation Plan....33
5.3 Coding....33
5.4 Testing....41
5.5 Installation....44
5.6 Maintenance....45

5.7 Conclusion....45
Chapter 6: Conclusion....46
6.1 Future Plans....46
6.2 Conversion Plan....46
6.3 Limitations....47
6.4 Recommendations....47
Bibliography....48
Appendix A: User Manual....50
A.1 Opening the platform....50
A.2 Choosing a lesson....50
A.3 Reading the notes....50
A.4 Guided coding....51
A.5 Do it yourself and Debug....52
A.6 Using a phone....53
A.7 If something does not work....53
Appendix B: Interview Checklist....54
B.2 Interview responses register....55
Appendix C: Questionnaire....58
C.2 Questionnaire responses register....59
Appendix D: Observation Score Sheet....62
Appendix E: Snippets of Code....63
E.1 Guided typing: accepting the expected key (shared-theme.js)....63
E.2 Challenge checker (shared-theme.js)....63
E.3 SQL parser: the SELECT statement (sql-model.js)....63
E.4 SQL engine: running a query (sql-model.js)....63
E.5 Scene: the timeline runner (sql-world.js)....64
E.6 Scene: table and rows built from data (sql-kit.js)....64
Appendix F: Lesson Content Index....65

LIST OF ACRONYMS

Acronym	Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DOM	Document Object Model
ER	Entity Relationship
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
LADO	Learn As you DO
LAN	Local Area Network
LLM	Large Language Model
PNG	Portable Network Graphics
S.M.A.R.T	Specific, Measurable, Achievable, Relevant, Time-bound
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SVG	Scalable Vector Graphics
UML	Unified Modeling Language
URL	Uniform Resource Locator
WebGL	Web Graphics Library

LIST OF TABLES

Table 1.1: Structure of the organisation relevant to the project....2
Table 1.2: Functions of the current system by department....5
Table 1.3: Comparison of existing solutions with the gaps found....6
Table 2.1: Inputs of the current system....8
Table 2.2: Deficiencies of the current system....9
Table 2.3: Advantages and disadvantages of Alternative 1....11
Table 2.4: Advantages and disadvantages of Alternative 2....12
Table 2.5: Business value of each alternative....12
Table 2.6: Estimated yearly costs and benefits (USD)....13
Table 2.7: Research methods, where they were used and why....15
Table 3.1: S.M.A.R.T objectives of the new system and their status....16
Table 3.2: Functions of the new system....17
Table 3.3: Inputs of the new system....18
Table 3.4: Deficiencies and limits of the new system....19
Table 3.5: Functional requirements and their status....19
Table 3.6: Non-functional requirements....20
Table 4.1: Process specification tools and the reason for choosing each....22
Table 4.2: Decision table for the message after a check....26
Table 4.3: Hardware and software requirements....29
Table 4.4: Data dictionary of the main data structures....31
Table 5.1: Implementation plan....33
Table 5.2: Main source files and their size....33
Table 5.3: How the code shows the required characteristics....36
Table 5.4: Quality features of the program....36
Table 5.5: Test results: SQL engine....41
Table 5.6: Test results: C++ teaching interpreter....42
Table 5.7: Maintenance plan....45
Table 6.1: Conversion plan....46
Table B.1: Interview checklist....54
Table B.2: Legend of the interview questions....55
Table B.3: Interview responses register, questions 1 to 5 (sample data)....56
Table B.4: Interview responses register, questions 6 to 10 (sample data)....57
Table C.1: Student questionnaire....58
Table C.2: Questionnaire responses register (sample data)....59
Table C.3: Summary of the questionnaire scores (sample data)....61
Table D.1: Observation score sheet (score 1 = poor, 5 = excellent)....62
Table F.1: Content of the three environments....65

Table F.2: Topics of the SQL environment and their scenes.....65

LIST OF FIGURES

Figure 1.1: Current workflow for practising programming....4
Figure 2.1: Context diagram (level 0) of the current system....9
Figure 2.2: Level 1 data flow diagram of the current system....10
Figure 2.3: Level 2 data flow diagram of process 3.0 (Practise on lab PC)....11
Figure 2.4: Work plan (Gantt chart) in weeks....14
Figure 3.1: Use case diagram of the new system....21
Figure 4.1: Context diagram (level 0) of LADO....22
Figure 4.2: Level 1 data flow diagram of LADO....23
Figure 4.3: Level 2 data flow diagram of process 5.0 (Check challenges)....24
Figure 4.4: Flowchart of guided typing....25
Figure 4.5: Flowchart of recording, previewing and replaying a program....25
Figure 4.6: UML class diagram of the 3D worlds....27
Figure 4.7: Sequence of messages from a key press to the result on screen....27
Figure 4.8: Architecture of LADO....28
Figure 4.9: Physical design: software and hardware interaction....29
Figure 4.10: Entity relationship diagram of the lesson content....30
Figure 4.11: Entity relationship diagram of the SQL engine and its recorded frames....31
Figure 5.1: Topic cards of the C++ environment....37
Figure 5.2: Notes card for a C++ subtopic....37
Figure 5.3: Guided typing of a function with the live 3D scene and output computer....38
Figure 5.4: SQL: CREATE TABLE builds the table column by column....38
Figure 5.5: SQL: DELETE sends a bin to the matching rows....39
Figure 5.6: SQL: a join links matching rows and the result appears on the computer....39
Figure 5.7: A Do it yourself task with the live goal checklist....40
Figure 5.8: Topic cards of the C# environment....40
Figure 5.9: Phone layout: topics, lesson menu and coding sheet....41
Figure A.1: Home page with the language buttons....50
Figure A.2: A notes page....51
Figure A.3: Guided coding with the robot copying a table to the screen....52
Figure A.4: A task with the goal checklist....53

LIST OF APPENDICES

Appendix A: User Manual.....50
Appendix B: Interview Checklist.....54
Appendix C: Questionnaire.....58
Appendix D: Observation Score Sheet.....62
Appendix E: Snippets of Code.....63
Appendix F: Lesson Content Index.....65

Chapter 1: Project Identification

1.0 Introduction

This document reports the development of LADO (Learn As you DO), a web-based learn-by-doing platform that teaches programming and database skills through simulated practice. Instead of reading about C++, C# or SQL, the learner types real code and immediately watches a three-dimensional scene build itself from that code: a table is assembled column by column, a bin walks over to take away deleted rows, and a robot copies the answer of a query onto a computer screen. Each lesson also contains simple-English notes, guided typing, and Do it yourself and Debug challenges that are checked automatically.

The report follows the National Diploma in Information Technology systems development project guideline. Chapter 1 identifies the organisation and the system under study. Chapter 2 presents the feasibility study. Chapter 3 analyses the new system. Chapter 4 describes its design. Chapter 5 explains the implementation, testing, installation and maintenance. Chapter 6 concludes the report with future plans, the conversion plan, limitations and recommendations. A bibliography in Harvard style and the appendices complete the document.

The original project proposal (Jakazira, 2026) described a platform covering C#, C++, SQL and Linux with an AI tutor, a scoring engine and a question-paper converter. During development the scope was refined so that the learning experience could be completed to a high standard for C++, C#, and SQL. Chapter 3 and Chapter 6 state clearly which proposal features were delivered, which were reduced, and which remain future work.

1.1 Concerned Organisation

The system is built for Masvingo Polytechnic, a technical and vocational tertiary institution that teaches programming and databases to diploma-level students. The organisation is summarised below.

1.1.1 Background

Masvingo Polytechnic is a state technical and vocational tertiary institution located in Masvingo, Zimbabwe. It offers national certificate and national diploma programmes in several technical and business fields, including Information Technology and Software Development. Students are taught programming languages and database design in

lectures and practise in computer laboratories. The Information Technology department is the department that this project was carried out for.

1.1.2 Line of business

The organisation's line of business is education and training. It enrols students, delivers lectures and practical sessions, assesses students through assignments, tests and examinations, and awards qualifications.

1.1.3 Structure

The institution is organised into academic departments that report to the principal. The department that uses the system under study is the Information Technology department, headed by its Head of Department. Lecturers and laboratory technicians report to the head of department, and students are grouped into classes by year of study.

Table 1.1: Structure of the organisation relevant to the project

Level	Role	Relation to the system
Principal	Overall management of the institution	Approves use of new teaching tools
Head of Information Technology department	Manages lecturers, curriculum and laboratories	Sponsors the project, receives progress reports
Lecturers (programming and databases)	Teach theory, set practical work, mark	Educators who supply lesson and challenge content
Laboratory technician	Maintains computers and software	Installs the system link or shortcut on lab computers
Students	Learn and practise	Learners who use LADO

1.1.4 Functions

- Teaching: lectures, tutorials and demonstrations in the classroom.
- Practical training: supervised laboratory sessions where students write programs and queries.
- Assessment: assignments, class tests, projects and examinations.
- Student support: consultation, feedback and remedial lessons.
- Administration: registration, timetabling and records.

1.1.5 Aims and objectives

The aim of the organisation, in relation to this project, is to produce graduates who can write working programs and database queries. The objectives below are S.M.A.R.T and are expressed for the Information Technology department.

1. To raise the amount of supervised-quality practical coding practice each student can do per week to at least four sessions of 30 minutes by the end of the next academic term, measured by the number of completed challenges per student.
2. To make instant feedback available on every practical exercise without a lecturer present, so that at least 80 percent of submitted practice attempts receive feedback within five seconds.
3. To cover every topic of the first-year programming and database syllabus with at least one guided example and three challenges by the end of the project period.
4. To make the practice tools usable on the students' own phones and laptops at no licence cost to the institution.

1.1.6 Vision

The vision adopted for this project is: "A polytechnic where every Information Technology student practises coding every day, understands what the computer does with each line they write, and graduates ready to build working software for Zimbabwe and beyond."

1.1.7 Mission statement

The mission adopted for this project is: "To provide free, simple and visual practice tools that let every student learn programming and databases by doing, with instant feedback on every attempt, so that Masvingo Polytechnic produces confident and competent software developers."

1.2 Current System Overview

1.2.1 Background of the system relating to the background of the organisation

Students at the organisation currently learn programming through lectures, slides, printed notes and textbook exercises, followed by laboratory sessions in which they install or open a compiler, an integrated development environment or a database tool and try to solve set exercises. Laboratory time is limited and is shared between many students, so most of the practice that students do outside lectures happens alone, on their own devices,

with no one to explain an error message. Because the organisation's mission is to produce practically competent graduates, the way practice is organised is the most important weakness of the present system.

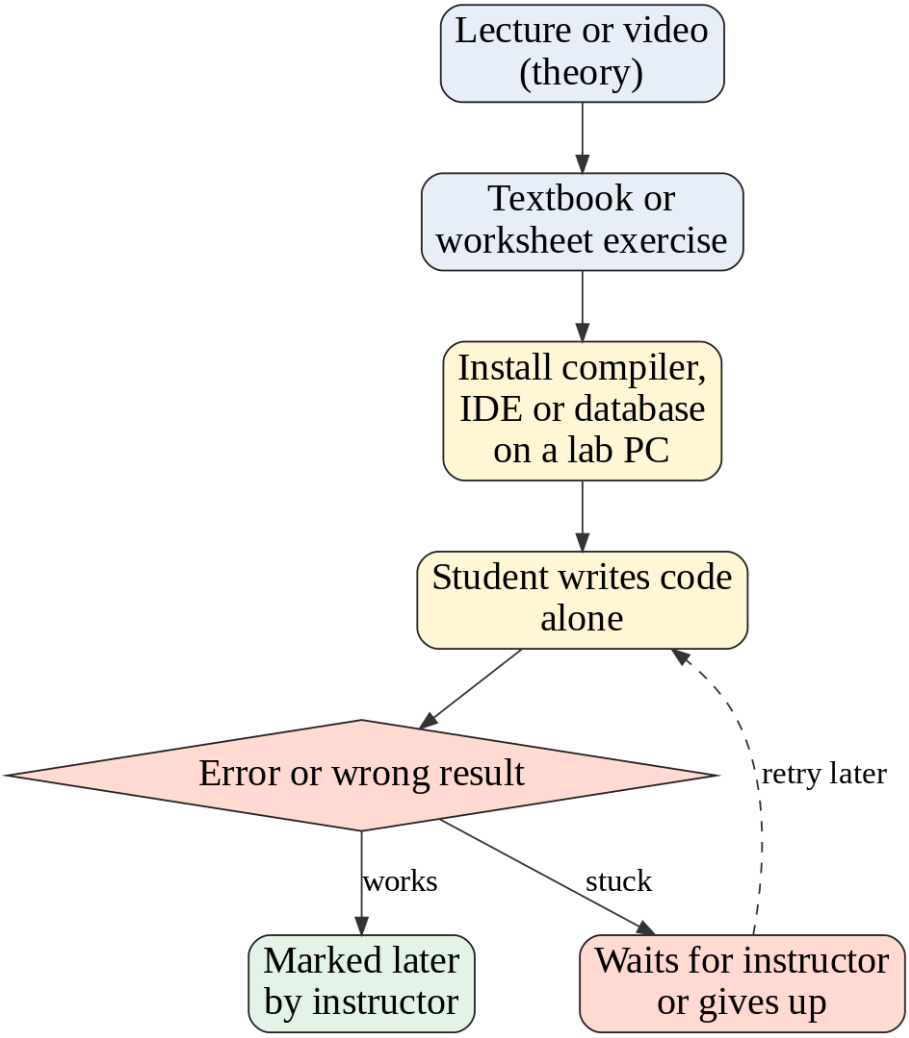


Figure 1.1: Current workflow for practising programming

1.2.2 System aims and objectives

The current practical learning system aims to help students apply what they were taught in lectures. Its objectives, stated in S.M.A.R.T form, are:

- Specific: every practical session should end with each student having written and run code for the topic of the week.
- Measurable: the lecturer marks the exercises and records the mark in the marks register.
- Achievable: only if each student has a working computer and software installed.
- Relevant: the exercises follow the syllabus.

- Time-bound: exercises are set weekly and marked before the next lesson.

1.2.3 System functions (and the departments in which they fall)

Table 1.2: Functions of the current system by department

Function	Department	Description
Present theory	Information Technology (lecturers)	Lectures, slides, notes
Set exercises	Information Technology (lecturers)	Textbook questions, worksheets, assignments
Provide practice machines	Computer laboratory (technician)	Install and maintain compilers and database tools
Practise coding	Students	Write and run programs in the laboratory or at home
Mark and give feedback	Information Technology (lecturers)	Marking, marks register, comments
Record results	Administration / Examinations office	Store marks and report to students

1.2.4 System justification

The system exists because practical skill cannot be acquired by listening alone. Active learning, in which students do something and think about what they are doing, produces better learning than passive instruction (Prince, 2004). The current system provides the setting for practice, but it provides little feedback while the student is working, which is the moment when feedback is most useful.

1.2.5 Problem definition

Practice is limited by laboratory time and by the availability of a person to explain mistakes. The specific problems are:

1. Students receive very little structured, hands-on practice compared with the amount of theory they are taught.
2. When a student gets stuck alone, an error message such as “expected ;” does not explain what the program is doing wrong, so the student guesses, copies a solution, or gives up.
3. Practice requires software to be installed on a computer, which is difficult for students who share devices or use phones.
4. Lecturers cannot easily turn an existing exercise into practice that corrects itself.

5. Students cannot see what the computer does with their code; variables, loops and tables stay invisible, so misconceptions remain.

1.2.6 Literature review

Research on computer science education supports the idea behind LADO. Sorva (2013) explains that novices struggle because they do not have a correct mental model of the "notional machine" that executes their program, and that visualisations of program execution help them to build one. Guo (2013) built the Online Python Tutor, which draws the state of a running program step by step, and reported use by very large numbers of learners worldwide. Ben-Ari (2001) argued that learners construct their own understanding and that this understanding is often wrong unless they can test it against a model that responds. VanLehn (2011) reviewed tutoring research and found that step-by-step, individual feedback approaches the effectiveness of a human tutor. Deterding et al. (2011) defined gamification as the use of game-design elements in non-game contexts, such as progress indicators and rewards, to increase engagement.

Mayer (2009) shows that people learn better when words are combined with well-designed pictures and when unnecessary decoration is removed. Sweller (1988) explains that learning suffers when working memory is overloaded. These ideas influenced the design of LADO's scenes: each command has one small animation that shows only what that command does, and the animations are kept slow, smooth and minimal.

Table 1.3: Comparison of existing solutions with the gaps found

Solution type	Strength	Gap for the organisation
Video lectures and slides	Easy to distribute	Passive; no practice and no feedback
Local compilers and IDEs	Real tools, full power	Installation, unclear errors, nothing shows what the code does
Online coding platforms (for example Codecademy, freeCodeCamp)	Interactive exercises, gamified	Generic; not matched to the local syllabus; limited free content; no visual model of execution
Program visualisers (for example Online Python Tutor)	Shows execution state	Covers one language; does not teach SQL or object-oriented topics through physical metaphors; no guided typing or challenges

1.2.7 New system alternatives and proposals

Three alternatives were considered:

- Alternative 1: keep the current system and add more laboratory hours and videos.
- Alternative 2: a server-based platform that compiles and runs students' code on real compilers in containers, with an AI tutor.
- Alternative 3: a browser-only platform that runs small teaching interpreters on the learner's device and visualises the program in 3D (LADO).

Alternative 3 was proposed and built, for reasons given in Chapter 2.

1.2.8 Conclusion

The current system gives students theory and a place to practise but not the guidance and visible feedback that make practice effective. A learn-by-doing platform that is free to run, works on any device and shows what code does is therefore justified. The next chapter examines the current system in detail and tests the feasibility of the alternatives.

Chapter 2: Feasibility Study

2.1 Introduction

This chapter describes the current system in detail, shows how data moves through it, lists its deficiencies, and compares the alternatives. For each alternative the technical, economic, social and operational feasibility are assessed, a recommendation is made, and a work plan is given. The chapter ends with the research methodology used to gather facts.

2.2 Current System Detail

2.2.1 Business procedures

1. The lecturer presents a topic in a lecture using slides and notes.
2. The lecturer gives an exercise sheet or an assignment on the topic.
3. Students go to the computer laboratory, or use their own device, and open a compiler, IDE or database tool.
4. Each student writes code and runs it. If an error appears, the student tries to interpret it alone or asks the lecturer or a classmate.
5. Students submit the work. The lecturer marks it, records the mark in the marks register and returns comments later.

2.2.2 Inputs

Table 2.1: Inputs of the current system

Input	Source	Form
Lesson plan and slides	Lecturer	Electronic files, printed notes
Exercise questions and marking scheme	Lecturer	Worksheets, textbooks
Student code and queries	Student	Source files, text files, database files
Questions during practice	Student	Spoken or written

2.2.3 Outputs

- Program output on the screen (only the student sees it).
- Compiler or database error messages.
- Marks and written comments from the lecturer, usually days later.

- Class marks reports from the marks register.

2.2.4 Security

- Laboratory computers are protected by user accounts and physical access control.
- Student work saved on shared computers can be viewed or deleted by other users when it is not protected.
- Marks are protected by the lecturer's copy of the register.

2.2.5 Controls

- Attendance registers control who is present in practical sessions.
- Marking schemes control how work is marked.
- Timetables control laboratory use.
- Moderation of marks by the head of department.

2.2.6 Current system deficiencies

Table 2.2: Deficiencies of the current system

Area	Problem experienced
Procedures	Practice depends on laboratory time and on a lecturer being available.
Inputs	Students must install and configure tools before they can type their first line.
Outputs	Error messages are technical and do not explain the cause; nothing shows what the program does internally.
Security	Work saved on shared machines can be lost or copied.
Controls	Feedback is late, so mistakes are practised and repeated before they are corrected.

2.2.7 Current system data flow diagrams up to second level

Figure 2.1 shows the context diagram. Figure 2.2 is the first-level diagram that breaks the system into its main processes. Figure 2.3 is the second-level diagram of process 3.0, "Practise on lab PC", where the problem occurs.

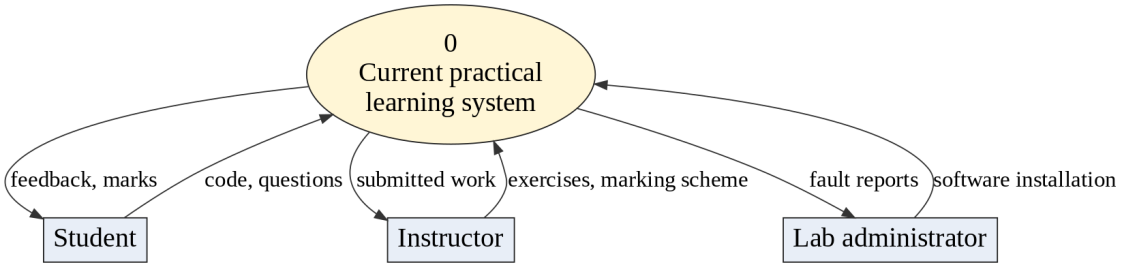


Figure 2.1: Context diagram (level 0) of the current system

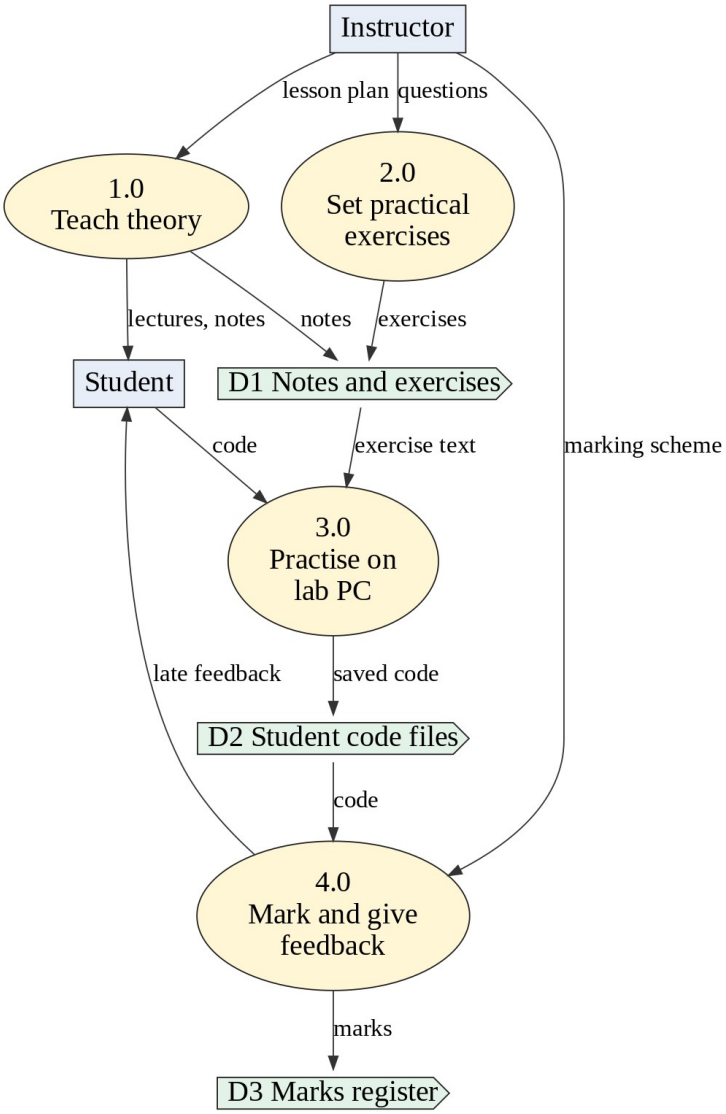


Figure 2.2: Level 1 data flow diagram of the current system

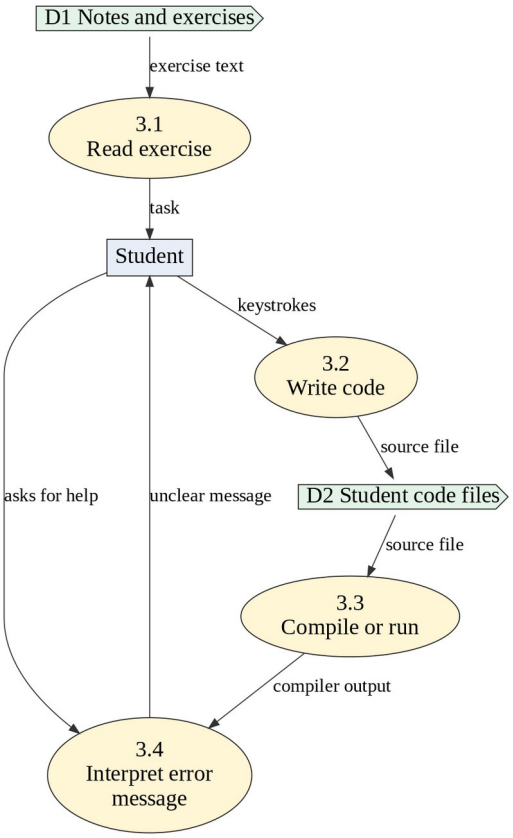


Figure 2.3: Level 2 data flow diagram of process 3.0 (Practise on lab PC)

2.3 Description of System Alternatives

Alternative 1: Server-based platform with real compilers and an AI tutor

Learners write code in the browser. The code is sent to a server that compiles and runs it in an isolated container and returns the output. A large language model answers questions about errors. User accounts, a database and a scoring engine record every attempt. This is the model described in the original proposal.

Table 2.3: Advantages and disadvantages of Alternative 1

Advantages	Disadvantages
Runs real compilers, so behaviour is exactly like industry tools	Needs a server, containers and a database that must be paid for and maintained
Can support any language that can be installed	Running student code safely needs careful sandboxing against abuse
Accounts allow persistent scores and class dashboards	Every run costs server time; cost grows with the number of learners
AI tutor can answer free questions	Requires internet and recurring API fees; answers can be wrong; data privacy concerns

Advantages	Disadvantages
	Shows only text output, so learners still cannot see what the code does

Alternative 2: Browser-only platform with teaching interpreters and 3D visualisation (LADO)

The whole platform is a set of static files. Small interpreters written in JavaScript run the learner's code in the browser and record every step. A three-dimensional scene replays those steps. Lessons, notes and challenges are stored in files. There is no server code and no database.

Table 2.4: Advantages and disadvantages of Alternative 2

Advantages	Disadvantages
No server to run: can be hosted free as static files	Interpreters support a teaching subset of each language, not the full language
Code runs on the learner's device, so there is no sandbox abuse risk and no run cost	No central accounts, so progress is not stored across devices yet
Works offline once loaded, on phones and laptops	Needs a device with WebGL for the 3D scenes (a flat fallback is provided)
Learners see what code does, which builds correct mental models	Every new language feature must be added to the interpreter by a developer
Instant feedback: results in milliseconds	No free-form AI tutor; hints are written in advance

2.3.1 Justification for building each alternative

Alternative 1 is justified by its realism and by its ability to scale to many languages. Alternative 2 is justified by its low cost, its immediate feedback and its ability to show the inside of the computer, which is the biggest learning gap identified in Chapter 1.

2.3.2 Business value of each alternative

Table 2.5: Business value of each alternative

Value measure	Alternative 1	Alternative 2
Initial cost	High (server set-up, containers, database)	Very low (static hosting)
Running cost	Recurring: hosting plus AI fees	Near zero
Time to first lesson	Long	Short

Value measure	Alternative 1	Alternative 2
Learning value (seeing the program run)	Low	High
Scalability to more learners	Costs grow with learners	No extra cost per learner
Risk	Security and cost risk	Scope risk (language subset)

2.4 Feasibility of Each Alternative

2.4.1 Technical feasibility

Alternative 1 requires skills in server administration, container security, a database and API integration. These are beyond what can be completed and secured within a diploma project period. Alternative 2 uses technologies that the developer already uses: HTML, CSS, JavaScript modules and the open-source Three.js graphics library. Browsers on laptops and phones support WebGL, so the 3D scenes run without installation. Alternative 2 is technically feasible, and a working prototype of it was built and tested.

2.4.2 Economic feasibility (cost-benefit analysis)

Table 2.6: Estimated yearly costs and benefits (USD)

Item	Alternative 1	Alternative 2
Development labour (student project)	0	0
Hosting	240 (small cloud server)	0 (GitHub or school server)
Container runners and database	240	0
AI service (for 100 active learners, estimate)	300	0
Domain name (optional)	10	10
Total yearly cost	790	10
Benefit: practice sessions without a lecturer	High	High
Benefit: visual understanding	Low	High

The costs are estimates based on typical prices of small cloud servers and usage-based AI services. Alternative 2 has a clearly better cost-to-benefit ratio for an institution with a limited budget.

2.4.3 Social feasibility

Students already use phones and laptops and are used to games and animations. A visual, step-by-step platform in simple English fits their habits and lowers the fear of programming. Lecturers keep their role: they write the content and decide how the platform is used. There is no threat to jobs, and no personal data is collected, so privacy concerns are minimal.

2.4.4 Operational feasibility

Learners need only a web browser and a link. The interface follows the same pattern for every language: choose a topic, read the notes, type along, try the challenges. Lecturers add content by editing plain text files. The laboratory technician has only to place a link on the computers. Operationally the platform is easy to use and to maintain.

2.4.5 Recommendations

Build Alternative 2, the browser-only teaching platform, first. Keep the design open so that a server, accounts and an AI tutor (Alternative 1 features) can be added later in a second phase when funding and time allow.

2.4.6 Work plan

Figure 2.4 shows the work plan in weeks, from analysis to documentation.

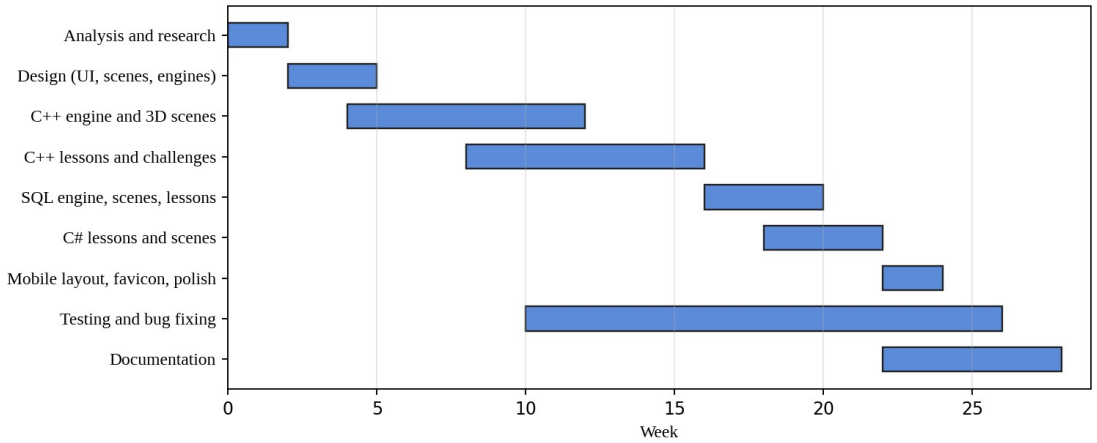


Figure 2.4: Work plan (Gantt chart) in weeks

2.4.7 Conclusion

The feasibility study shows that the browser-only teaching platform is technically, economically, socially and operationally feasible and is better suited to the organisation's resources than the server-based alternative.

2.5 Research Methodology

Several methods were used to gather facts. Each was chosen because it answers a different question.

Table 2.7: Research methods, where they were used and why

Method	Where it was used	Why it was chosen
Observation	Laboratory sessions and self-study time	To see how students actually behave when they are stuck, which cannot be learned from a questionnaire
Interviews	Lecturers and the laboratory technician	To learn about the syllabus, marking practice and software problems in depth (checklist in Appendix B)
Questionnaires	Students	To collect the opinions of many students quickly and compare answers (Appendix C)
Document review	Syllabus, textbooks and exercise sheets	To make sure the lessons cover the syllabus
Literature review	Journals and books	To base the design on research on learning (Chapter 1)
Prototype testing	Each lesson as it was built	To check that scenes, notes and challenges work (Chapter 5)
Observation score sheet	Learners using the prototype	To record how long tasks took and where users hesitated (Appendix D)

Chapter 3: Analysis Phase (For the New System)

3.1 Introduction

This chapter analyses the new system, LADO. It states the aims and objectives, the procedures, functions, inputs, outputs, security and controls of the new system, the deficiencies that remain, the research methods, and the requirements. The scope stated here is the scope that was actually built; Section 3.4 marks which requirements of the original proposal were met.

3.2 New System Details

3.2.1 System aims and objectives

The aim is to provide a free, browser-based learn-by-doing platform in which learners type code in C++, C# and SQL and see, in a 3D scene and on a screen, what the code does. The S.M.A.R.T objectives are:

Table 3.1: S.M.A.R.T objectives of the new system and their status

No.	Objective	Measure	Status
1	Build simulated browser-based practice environments for C++, C# and SQL	Three environments open from the home page and run code	Met: three environments built. Linux not built
2	Give every subtopic simple-English notes, 3 guided examples and challenges	106 subtopics, 329 guided examples, 237 challenges	Met
3	Give instant feedback with a live goal checklist, readable errors, hints and answers	Result of a check shown within one second	Met
4	Show what the code does in 3D, in step with the program	Scene built for each topic and replayed as the learner types	Met for C++ topics and SQL; C# reuses the C++ scenes
5	Work on phones as well as desktops	No sideways scrolling at 390 px width; menu and coding sheet usable	Met
6	Keep a score and progress record per learner	Persistent scores and dashboards	Not met in this version: moved to future work
7	Use an AI tutor and convert question papers to challenges	LLM-based feedback and converter	Not met in this version: hints are written in advance; moved to future work

3.2.2 Business procedures

1. The learner opens the home page and chooses a language: C++, C# or SQL.
2. The learner chooses a topic card and then a subtopic.
3. The learner reads the notes card: title, official definition, simple explanation, real-life example, code and further parts.
4. The learner presses Guided coding on an example. The editor is empty except for the boilerplate. The learner types the code with ghost text showing the next character.
5. As the learner types, the 3D scene builds. After a short pause the scene replays the whole program step by step and the computer screen shows the result.
6. The learner presses Do it yourself and solves tasks. A checklist of goals ticks live. When all goals are ticked, the task is marked correct.
7. In a Debug task, the learner finds and fixes the mistake in broken code. Hints and the full answer can be requested.

3.2.3 System functions

Table 3.2: Functions of the new system

Function	Description
F1 Lesson navigation	Topic cards, side menu, previous and next buttons, direct links
F2 Notes	Card-style notes in simple English for each subtopic
F3 Guided typing	Phased type-along with ghost text, error shake and automatic capital letters in SQL
F4 Teaching engines	C++ interpreter, C# translator, SQL engine record every step
F5 3D visualisation	Scenes built and replayed from the recorded steps
F6 Result screen	A computer screen shows output, result tables, messages and errors
F7 Challenges	Do it yourself and Debug tasks with goals, hints and answers
F8 Appearance	Dark and light themes; mobile-first layout; icons and favicon

3.2.4 System justification

The new system is justified because it removes the three main causes of the problem in Chapter 1: it needs no installation, it gives feedback within milliseconds, and it shows the learner what the code does. It also costs almost nothing to run.

3.2.5 Inputs

Table 3.3: Inputs of the new system

Input	Source	Validation
Typed code or SQL	Learner	Guided mode only accepts the expected next character; free mode accepts any text and reports errors
Button clicks and taps	Learner	Buttons disabled when they do not apply (for example Hint while a task is being completed)
Lesson content files	Educator	Content validators run every example and every challenge solution
Theme choice	Learner	Dark or light

3.2.6 Outputs

- A live 3D scene of the program.
- A computer screen with the program output, the result table of a query, a success message or an error.
- A goal checklist with ticks, a hint, the answer, and a praise message when a task is solved.
- Highlighted code with coloured keywords.

3.2.7 System security

- The platform has no server code and no database, so there is nothing to attack on the server side beyond the static files.
- Learner code is never sent anywhere: it runs in an interpreter inside the browser tab and cannot touch the file system or network.
- Endless loops and endless recursion are stopped by a step limit and a call-depth limit.
- No personal data is collected.

3.2.8 System controls

- Step limit and call-depth limit in the C++ engine.
- Content validators: every guided example must run, every challenge solution must pass, every Debug program must fail, and no Do it yourself task may pass with an empty answer.

- Constraint checks in the SQL engine (NOT NULL, UNIQUE, data type).
- Version tokens on every file so browsers always load the current version.

3.2.9 New system deficiencies

Table 3.4: Deficiencies and limits of the new system

Area	Deficiency
Procedures	Progress and scores are not saved between visits.
Inputs	Interpreters understand a teaching subset: pointers to pointers, templates and the full standard library are not supported.
Outputs	3D scenes need WebGL; the very slowest phones may animate less smoothly.
Security	No accounts, so an educator cannot see class results.
Controls	No automated tests yet for the C# environment; Linux environment not built.

3.3 Research Methodologies

The methods listed in Section 2.5 continued into the analysis phase. Questionnaires and interviews fixed the list of topics; observation of learners using early prototypes showed that long notes were skipped and that animations that move too much distract, which led to the rule of one small, slow animation per command. Prototype testing was repeated after every topic was built. The interview checklist (Appendix B) and the student questionnaire (Appendix C) are the instruments for lecturers and students of the Information Technology department of Masvingo Polytechnic. The registers in the appendices contain illustrative sample answers that show how responses are recorded and summarised; they are not collected data. The design decisions in this report, which are instant feedback, short notes in simple English, and a platform that works on phones, follow from the problem definition in Chapter 1, the literature reviewed, and observation of how students practise.

3.4 Requirements Analysis

Functional requirements

Table 3.5: Functional requirements and their status

ID	Requirement	Status
FR1	The system shall let a learner choose C++, C# or SQL	Done

ID	Requirement	Status
FR2	The system shall show notes for every subtopic in the agreed card layout	Done
FR3	The system shall let a learner type a guided example with a visible next-character hint	Done
FR4	The system shall run learner code in a teaching engine and record every step	Done (C++, C#, SQL)
FR5	The system shall build and replay a 3D scene from the recorded steps	Done
FR6	The system shall show output, result tables and errors on a computer screen	Done
FR7	The system shall provide Do it yourself and Debug tasks with live goals, hints and answers	Done
FR8	The system shall run on phones with a menu and a stacked coding sheet	Done
FR9	The system shall keep a score for each learner	Not done (future work)
FR10	The system shall provide a Linux terminal environment	Not done (future work)
FR11	The system shall provide an AI tutor and question-paper conversion	Not done (future work)

Non-functional requirements (constraints)

Table 3.6: Non-functional requirements

Quality	Requirement
Performance	A step of code must be recorded in under one second for lesson-sized programs; a scene frame must render smoothly at 30 frames per second or more on a mid-range laptop.
Usability	Notes in simple English; one animation at a time; large tap targets (at least 42 px) on phones.
Portability	Runs in current Chrome, Edge, Firefox and Safari without installation.
Availability	Works offline after the first load; hosted as static files.
Maintainability	Content in separate files; one module per environment; code and content validators.
Cost	No recurring cost beyond an optional domain name.

Security features

- Learner code runs only inside the interpreter in the browser tab.

- Execution limits stop endless loops and recursion.
- No cookies, accounts or personal data.

3.5 Conclusion

The analysis confirms that the core learning experience (notes, guided typing, 3D scenes, challenges) can be delivered for three languages with browser-only technology. Scores, accounts, a Linux environment, and an AI tutor are recorded as future work.

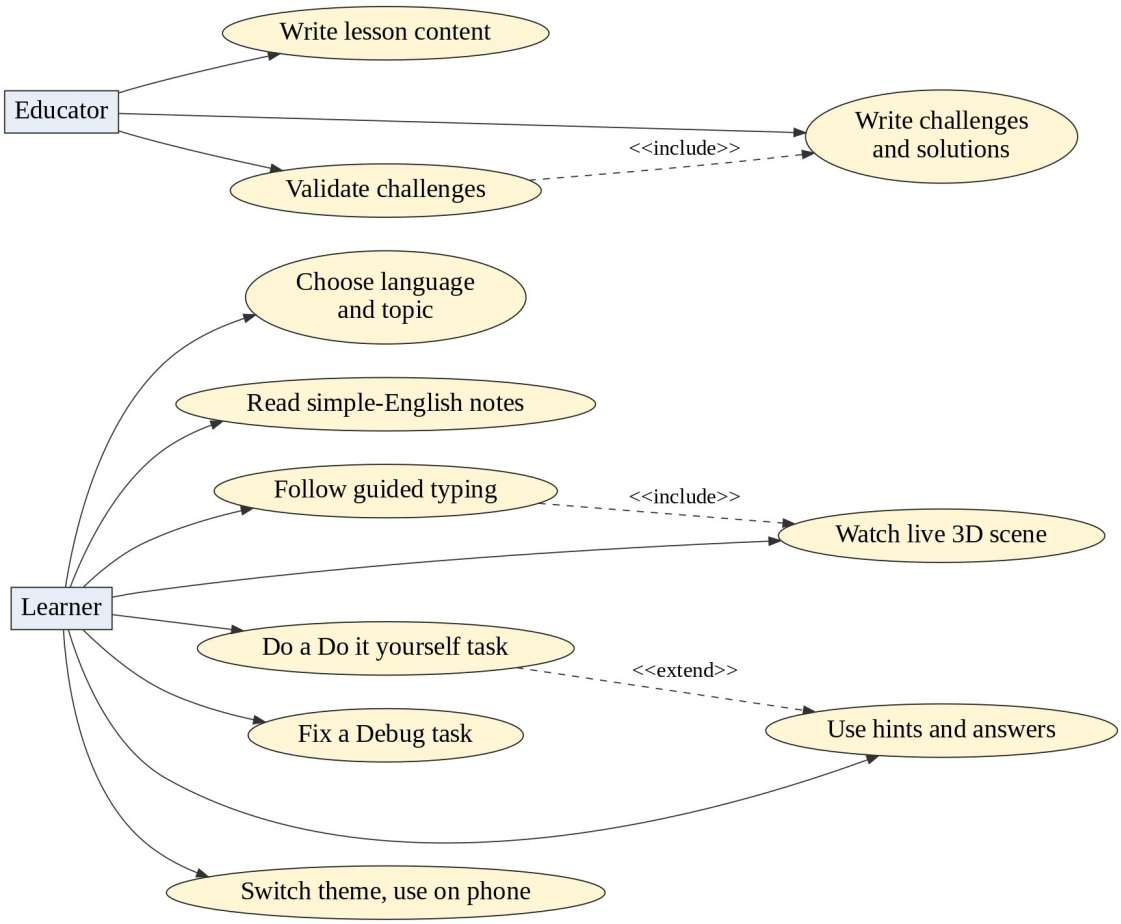


Figure 3.1: Use case diagram of the new system

Chapter 4: Design Phase

4.1 Introduction

This chapter explains how the system works. It presents the process specification with data flow diagrams, flowcharts, structured English, a decision table and UML diagrams; then the architectural and physical design; then the design of inputs, processes, outputs, controls, files, interface and data dictionary. The program design is object oriented where a problem has things that own data and behaviour (worlds, tables, rows), and functional where a task is a pure transformation (tokenising, running, comparing).

4.2 System Design

4.2.1 Process specification

The following tools were chosen, each for a reason:

Table 4.1: Process specification tools and the reason for choosing each

Tool	Used for	Reason
Data flow diagrams (levels 0 to 2)	How data moves between the learner, the engines and the stores	Shows the whole system and each process without code detail
Flowchart	Guided typing and the replay loop	These are step-by-step decisions with loops
Structured English	Challenge checking	Readable by lecturers who are not programmers
Decision table	Choosing the message shown after a check	Several conditions combine into different outcomes
Pseudo-code	Applying a recorded step to the scene	Close to the real code but independent of language
UML class and sequence diagrams	Modules of the 3D worlds and the flow from key press to screen	Show object-oriented structure and order of messages

Figures 4.1 to 4.3 show the data flow diagrams of the new system up to the second level.

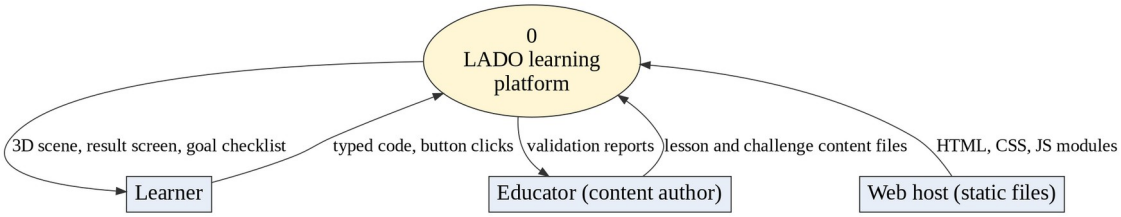


Figure 4.1: Context diagram (level 0) of LADO

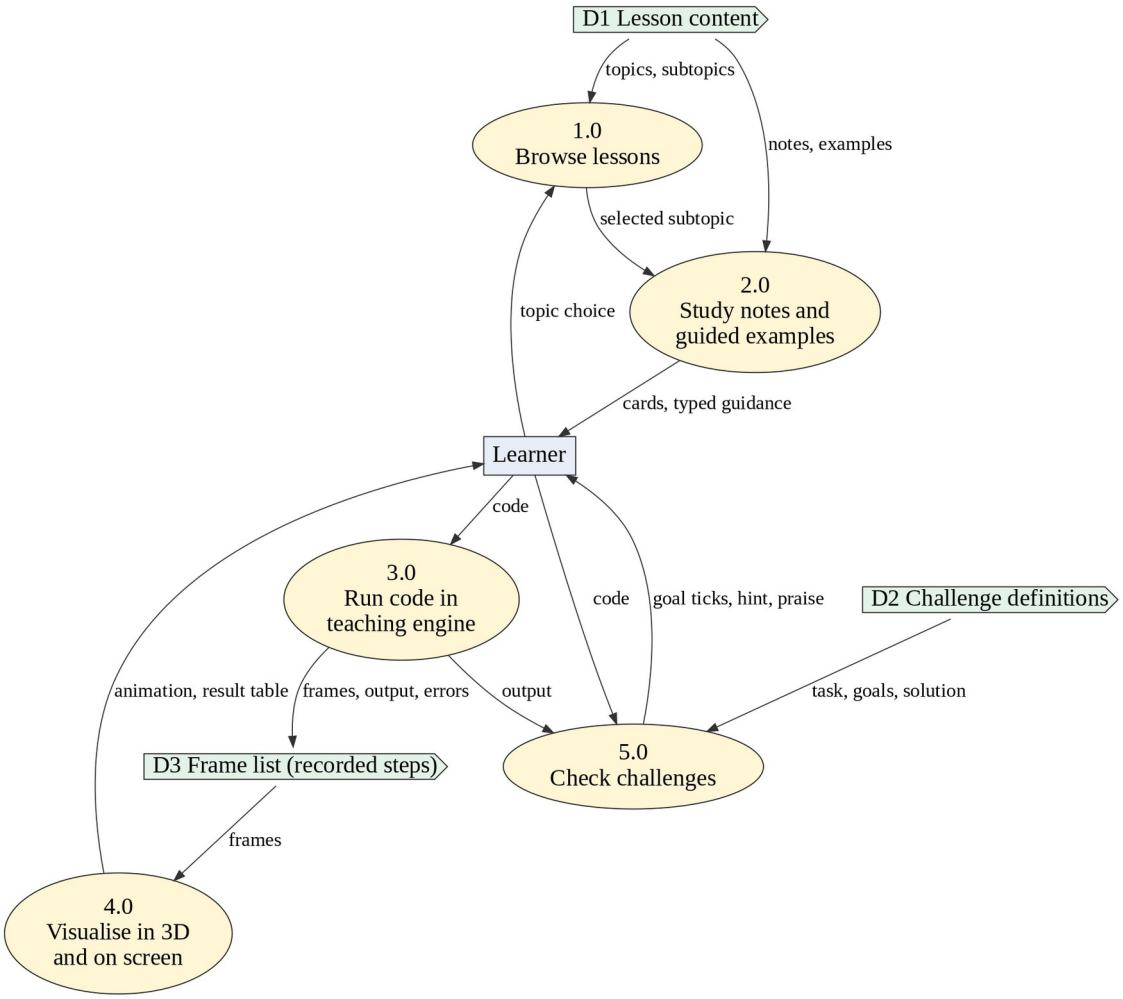


Figure 4.2: Level 1 data flow diagram of LADO

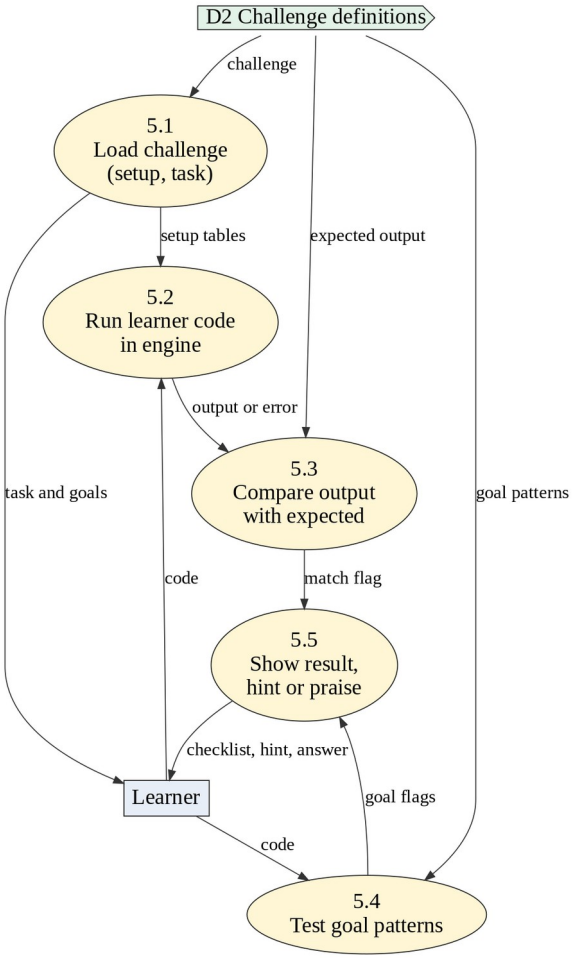


Figure 4.3: Level 2 data flow diagram of process 5.0 (Check challenges)

The flowcharts in Figures 4.4 and 4.5 describe the two most important procedures: guided typing and the replay of a program in the scene.

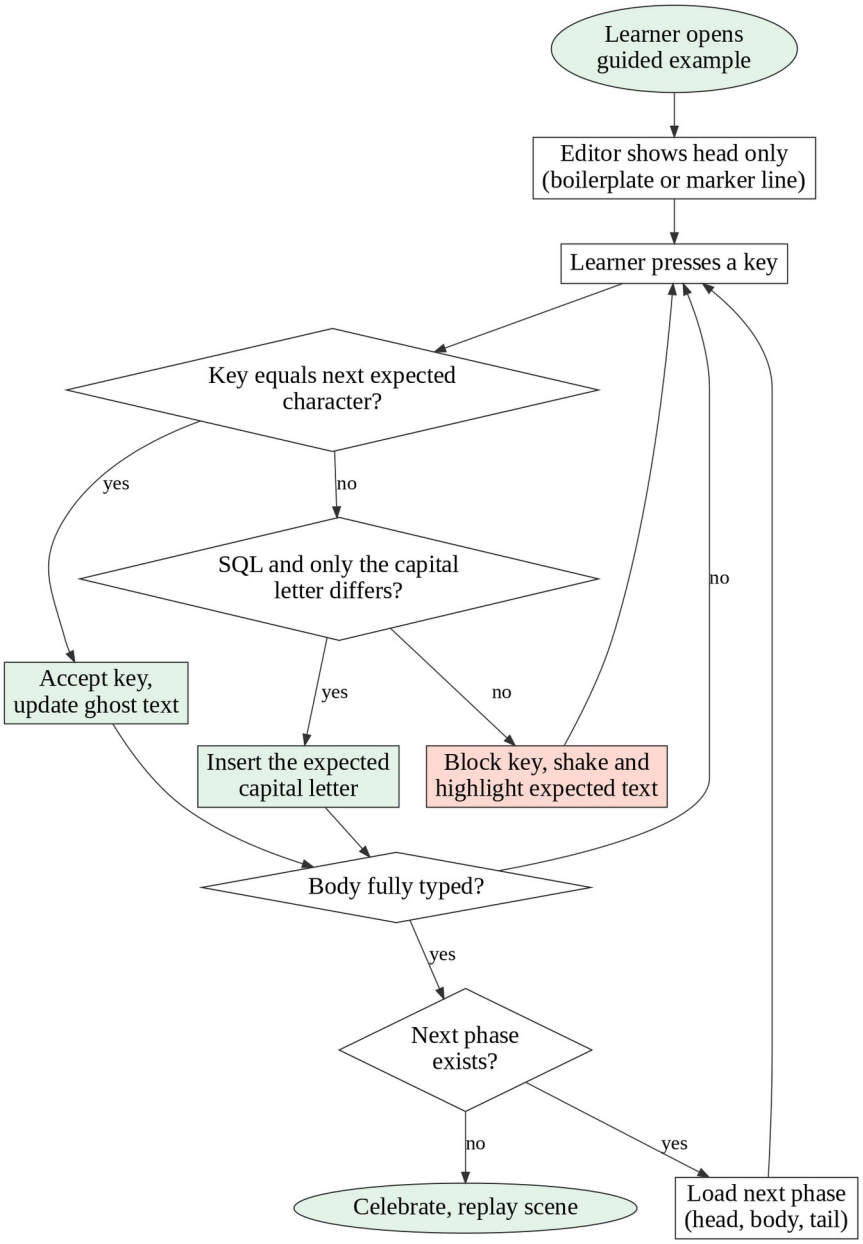


Figure 4.4: Flowchart of guided typing

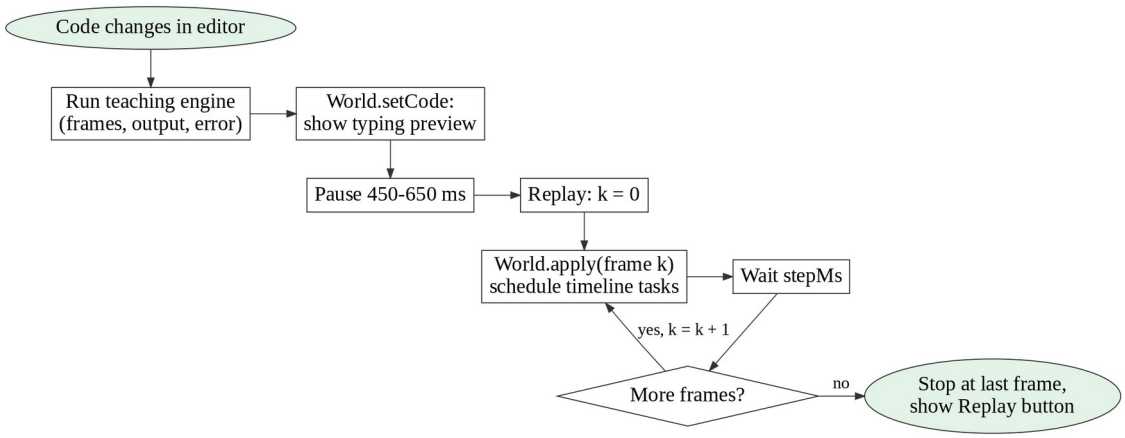


Figure 4.5: Flowchart of recording, previewing and replaying a program

Structured English: checking a challenge

```
WHEN the learner types or presses "Check my code":
  RUN the learner's code in the teaching engine (with the task's ready-
made tables, if any)
  IF the engine reports an error THEN
      goal "runs without an error" = NOT ticked
  ELSE
      goal "runs without an error" = ticked
  IF the output equals the expected output (ignoring spaces at line ends)
THEN
      goal "it shows ..." = ticked
  FOR EACH extra goal of the task:
      TEST the learner's code against the goal's pattern
      TICK the goal if the pattern is found (or not found, for "must not
use" goals)
  IF all goals are ticked THEN
      mark the task correct, show praise, load the next task
  ELSE
      show the first unmet goal, the engine error with its hint, or the
output difference
```

Decision table: message shown after "Check my code"

Table 4.2: Decision table for the message after a check

Rule	Engine error?	Output equals expected?	All goals ticked?	Message shown
1	Yes	-	-	The error and its hint
2	No	No	-	What your program printed and what it should print
3	No	Yes	No	Not yet: the first unmet goal
4	No	Yes	Yes	Correct, praise and next task

Pseudo-code: applying a recorded step to the 3D scene

```
PROCEDURE apply(frame, previousFrame, jump):
    finish any running animation
    IF previousFrame is missing OR jump THEN
        rebuild the scene instantly from frame.snapshot
        RETURN
    offset = 0
    FOR EACH event IN frame.events:
        offset = offset + handle(event, offset)      // schedules timed
tasks
    schedule at (offset + 40 ms): make the scene equal frame.snapshot
    stepTime = (offset + 500 ms) / speed
    screenTime = arrival time of the result pieces
END PROCEDURE
```

Where an object-oriented design was used, Figure 4.6 shows the class structure of the 3D worlds, and Figure 4.7 shows the order of messages from a key press to the result on

screen. All scene worlds inherit from a common World class. TimedWorld adds a timeline of timed tasks. SqlWorld, FileWorld, StructWorld and ClassWorld each extend TimedWorld with the animations of their own topic.

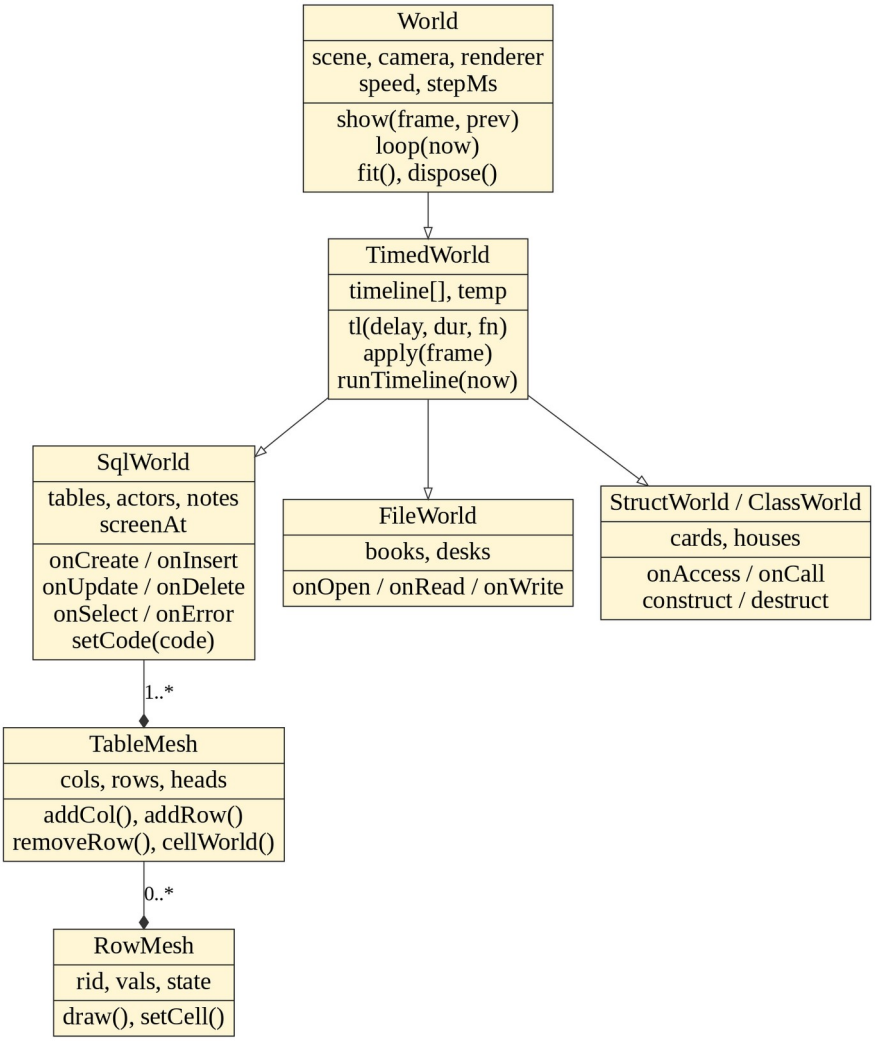


Figure 4.6: UML class diagram of the 3D worlds

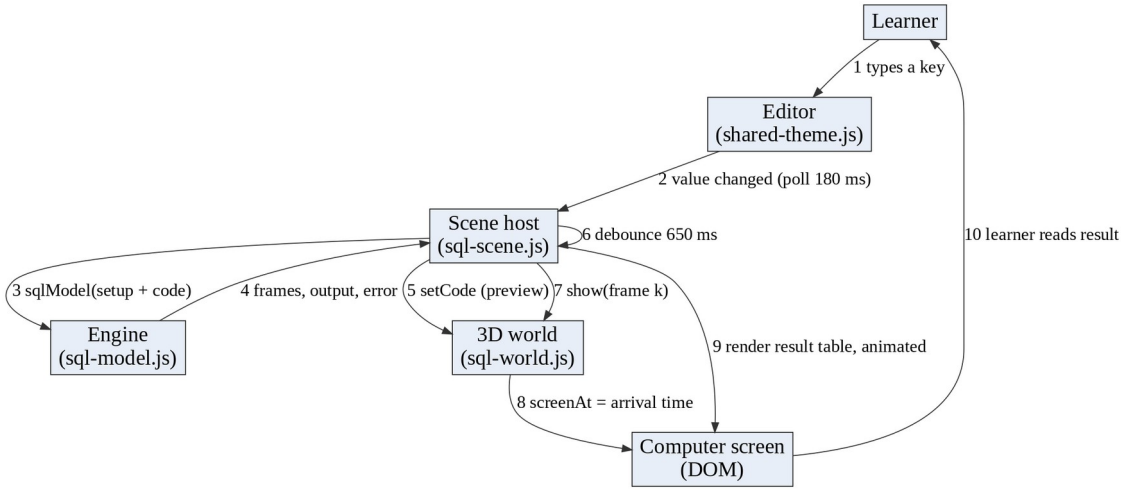


Figure 4.7: Sequence of messages from a key press to the result on screen

4.3 Architectural Design

LADO is a client-side application. All logic runs in the browser. The shell page (index.html) loads one environment module at a time. Each environment module collects its lesson content and challenges and passes them to a shared workspace (shared-theme.js) that draws the pages and runs the guided typing and challenge checker. A teaching engine records the steps of the learner's program. A scene host feeds those steps to a Three.js world. Layers depend only on the layer below them, which keeps coupling low.

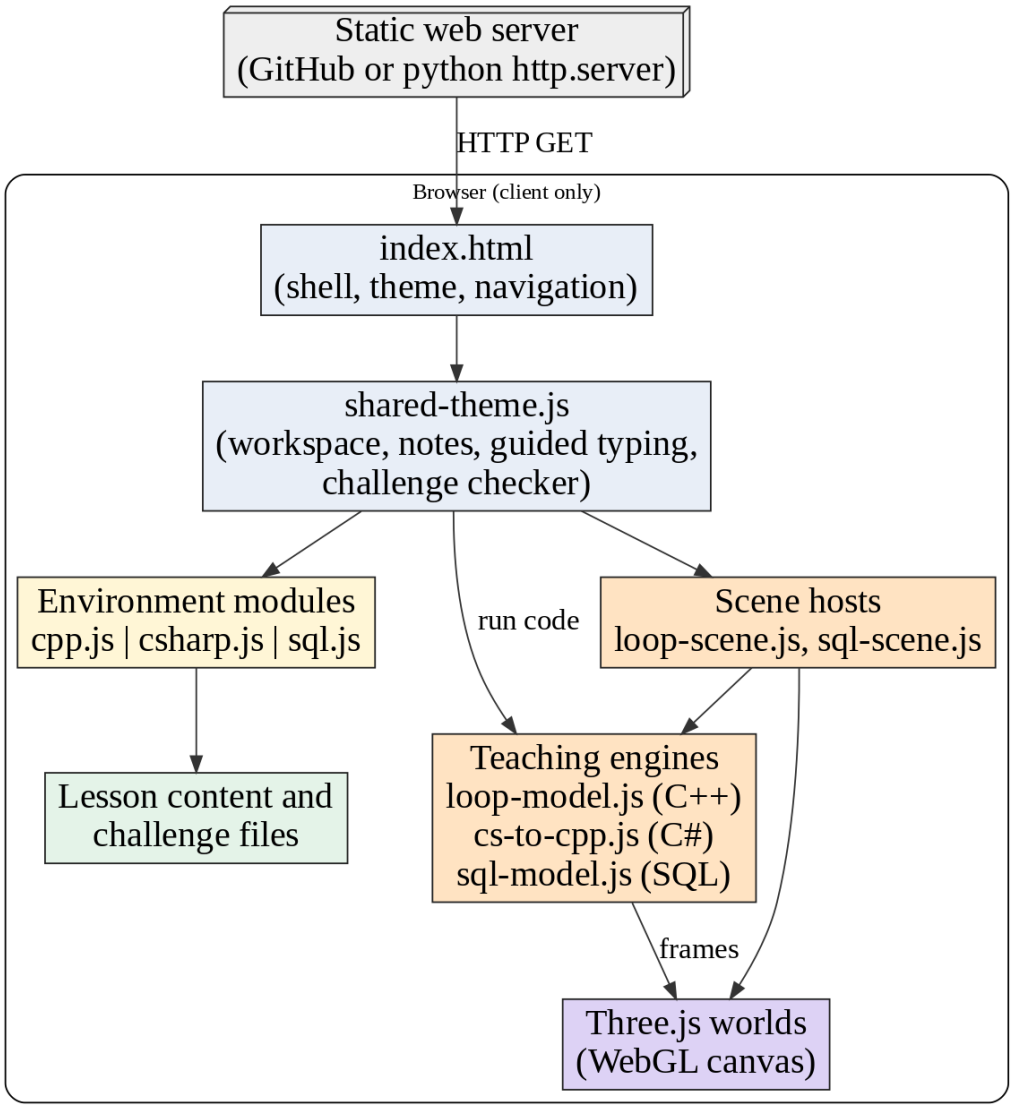


Figure 4.8: Architecture of LADO

4.4 Physical Design

Figure 4.9 shows how software and hardware interact. The developer works on a Linux laptop and pushes the project to a GitHub repository. The files are published on a static

web host. A learner's device downloads the files and runs everything locally; the graphics card draws the 3D scenes through WebGL.

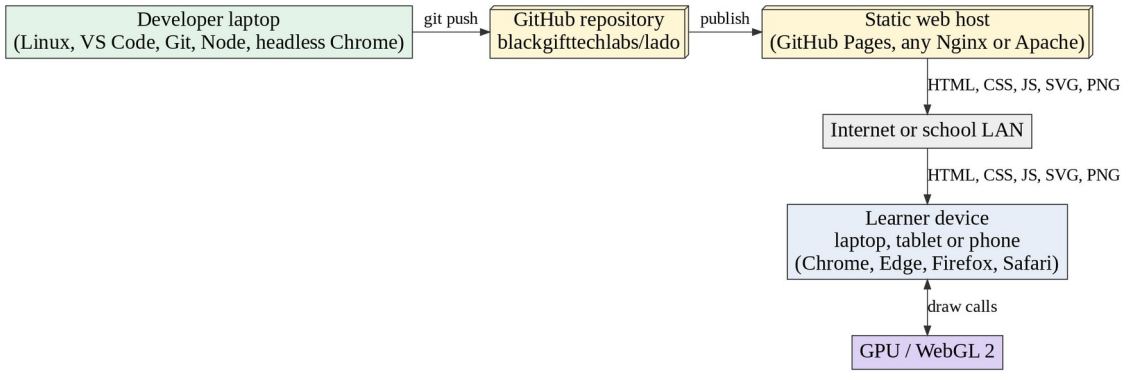


Figure 4.9: Physical design: software and hardware interaction

Table 4.3: Hardware and software requirements

Item	Minimum	Recommended
Learner device	Phone or laptop with 2 GB RAM and WebGL	Laptop with 4 GB RAM or more
Browser	Current Chrome, Edge, Firefox or Safari	Latest Chrome or Edge
Network	Needed once to load the files	School LAN or broadband
Server	Any static web server	GitHub Pages or Nginx
Developer tools	Text editor, Git, Python 3 (for a local server)	VS Code, Node.js, Chrome for tests

4.5 Design of Inputs, Processes, Outputs, Controls, Files and Interface

4.5.1 Input design

- Code editor: a text area with syntax colours drawn behind it. In guided mode it accepts only the expected next character.
- Buttons: Guided coding, Do it yourself, Check my code, Hint, Skip, Reset, Run code, Play, Previous, Next.
- Menu and cards: tap targets of at least 42 px on phones.

4.5.2 Process design

- Tokenise, parse and run the code (teaching engines).
- Record frames: one per statement or step, each with events and a snapshot of the state.

- Replay frames in the scene on a timeline.
- Check challenges against expected output and goal patterns.

4.5.3 Output design

- 3D scene in the upper part of the lesson sheet.
- A one-line banner stating what the current step does and the statement that is running.
- A computer screen below the scene: terminal text for C++ and C#, a result table or message for SQL.
- Goal checklist, hints and praise in the challenge bar.

4.5.4 System controls

- Step limit and call-depth limit.
- Content validators and a rule that content changes must pass them.
- Version tokens in all import paths.
- Typed constraint checks in the SQL engine with plain-English hints.

4.5.5 Files and database design

LADO has no database. Its data is stored in JavaScript files that hold the lesson content and the challenge definitions, and in memory while a program runs. Figure 4.10 shows the content data model as an entity relationship diagram. Figure 4.11 shows the in-memory data model of the SQL engine, which behaves as a small database during a run.

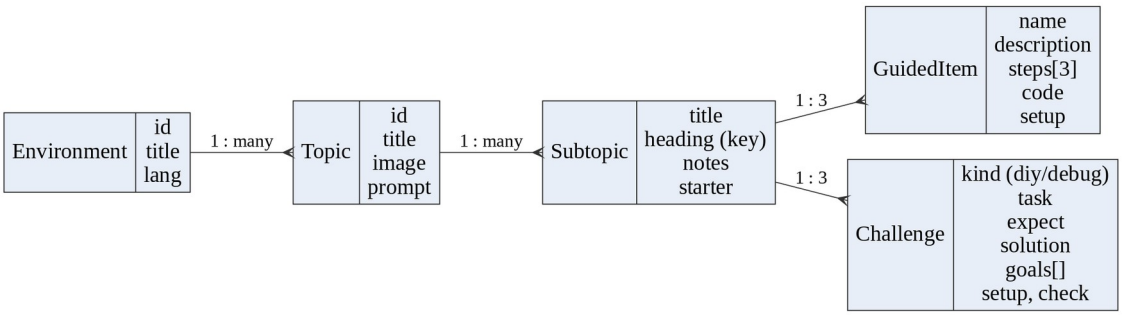


Figure 4.10: Entity relationship diagram of the lesson content

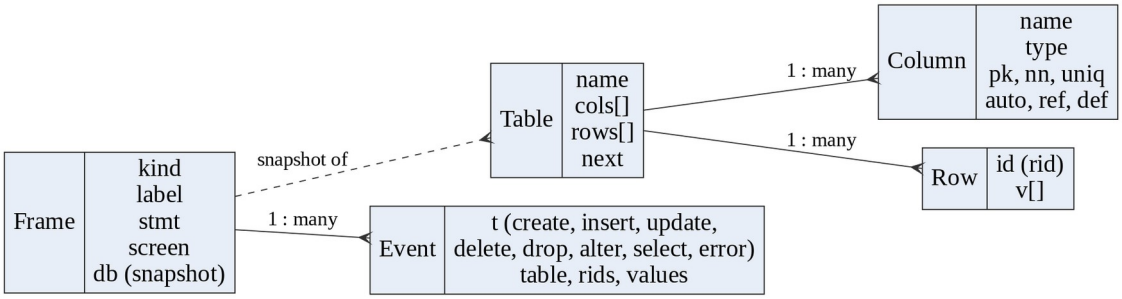


Figure 4.11: Entity relationship diagram of the SQL engine and its recorded frames

4.5.6 Interface design

Interface rules: one design for every language; dark and light themes; notes cards with a large uppercase title, a purple “Official definition” label, a tinted definition box and bordered bullet cards; the coding sheet shows the editor on the left and the scene and computer on the right on desktop, and stacks them on phones; the output computer is large and faces the learner. Screens of the finished interface are shown in Chapter 5 and Appendix A.

4.5.7 Data dictionary

Table 4.4: Data dictionary of the main data structures

Name	Type	Description
Topic	Object	id, title, image, prompt and a list of subtopics
Subtopic	Object	title, lang, content (heading, notes, items, practice)
Notes	Object	definition (text with bold markers), points (term and text), code
GuidedItem	Object	name, description, steps (3), code, optional setup, flags guided/full/warning
Challenge	Object	kind (diy or debug), task, expect, solution, goals (re or no pattern), hint, why, setup, check
Frame	Object	kind, label, stmt, events[], screen, snapshot (db or variables)
Event	Object	t (type) and the data that scene needs, for example table, rids, values
Table (SQL)	Object	name, cols[], rows[], next row id
Column (SQL)	Object	name, type, primary key, not null, unique, auto, reference, default
Row (SQL)	Object	id (row id) and v[] values
setup	String	SQL that builds the ready-made tables of an example or challenge
check	String	Hidden query (or @table / @tables) that shows what the learner’s SQL did

4.6 Conclusion

The design separates content, engines, scenes and the lesson workspace so that each can change without breaking the others. The next chapter shows how the design was implemented and tested.

Chapter 5: Implementation Phase

5.1 Introduction

This chapter explains how LADO was built and checked. It gives the implementation plan, samples of code and how the code meets the quality guidelines, the test plan with the results of real test runs, the installation and documentation, and the maintenance plan.

5.2 Implementation Plan

Table 5.1: Implementation plan

Stage	Work	Output
1	Shared lesson workspace: topic cards, notes cards, guided typing, challenge bar, themes	shared-theme.js and styles
2	C++ teaching interpreter and loop, branching and operator scenes	loop-model.js and Three.js worlds
3	C++ topics: arrays and strings, functions, structures, classes, files, each with a scene	8 topics, 61 subtopics
4	SQL engine and database scenes (tables, bin, robot, computer)	sql-model.js, sql-world.js, 4 topics
5	C# lessons that reuse the C++ scenes	csharp.js and 24 topics
6	Mobile-first layer, favicon, topic images	mobile.css, assets
7	Testing, validators and bug fixing	Test results (Section 5.4)
8	Publishing to a GitHub repository and documentation	Repository and this document

Development was done in small steps. After every step the lesson was opened in a real browser, typed through, and checked against the content validators before the next step started. The code is stored in the repository https://github.com/blackgifttechlabs/lado.

5.3 Coding

Size and organisation of the code

Table 5.2: Main source files and their size

File	Role	Lines
`index.html`	Shell page, themes, navigation	645
`shared-theme.js`	Shared lesson workspace	3554
`mobile.css`	Phone layout	61

File	Role	Lines
environments/cpp/cpp.js	C++ environment wiring	159
environments/cpp/topics/loop-model.js	C++ teaching interpreter	1473
environments/cpp/three/loop-worlds.js	World base class and loop scenes	451
environments/cpp/three/timed-world.js	Timeline base for object scenes	88
environments/cpp/three/struct-world.js	Structures scene	296
environments/cpp/three/class-world.js	Classes scene	204
environments/cpp/three/file-world.js	Files scene	233
environments/sql/sql.js	SQL environment wiring	89
environments/sql/topics/sql-model.js	SQL engine and parser	833
environments/sql/topics/sql-scene.js	SQL scene host and result screen	111
environments/sql/three/sql-world.js	SQL 3D animations	472
environments/sql/three/sql-kit.js	Tables, rows, tiles, bin and robot	273
environments/csharp/csharp.js	C# environment wiring	125
environments/csharp/topics/cs-to-cpp.js	C# to C++ translator	58

Altogether the authored HTML, JavaScript and CSS (without the vendor copy of Three.js) is about 24,000 lines. The content is 106 subtopics with 329 guided examples and 237 challenges.

Sample 1: the SQL engine tokenises text into tokens

```
function tokenize(src) {
  const out = []; let i = 0, line = 1;
  const push = (t, v, s, e, extra = {}) => out.push({ t, v, s, e, line, raw: src.slice(s,
e), ...ex…
  while (i < src.length) {
    const c = src[i];
    if (c === '\n') { line++; i++; continue; }
    if (/\s/.test(c)) { i++; continue; }
    if (c === '-' && src[i + 1] === '-') { while (i < src.length && src[i] !== '\n') i++;
continue;…
    if (c === '/' && src[i + 1] === '*') { const e = src.indexOf('*/', i + 2); i = e <
0 ? src.leng…
    const s = i;
    if (/\d/.test(c) || (c === '.' && /\d/.test(src[i + 1] || ''))) {
      while (/\d/.test(src[i] || '')) i++;
```

```
      if (src[i] === '.' && /\d/.test(src[i + 1] || '')) { i++; while (/\d/.test(src[i]
|| '')) i++…
      push('num', Number(src.slice(s, i)), s, i); continue;
    }
    if (c === "'") {
      i++; let v = '', closed = false;
      while (i < src.length) { if (src[i] === "'") { if (src[i + 1] === "'") { v += "'";
i += 2; co…
      push('str', v, s, i, { open: !closed }); continue;
    }
    if (c === '"' || c === '`') {
      const q = c; i++; let v = '';
```

Sample 2: the DELETE statement of the engine records an event

```
  doDelete(st, ev) {
    const t = this.table(st.name);
    const { scanned, matched } = this.matchRows(t, st.where);
    const ids = new Set(matched.map(r => r.id));
    ev.push({ t: 'delete', table: t.name, scanned, matched: [...ids], where: !!st.where,
all: ids.s…
    t.rows = t.rows.filter(r => !ids.has(r.id));
    return { msg: `${ids.size} row${ids.size === 1 ? '' : 's'} deleted from $
{t.name}.` };
```

Sample 3: the 3D scene turns the event into an animation

```
  onDelete(e, ofs) {
    const t = this.tbl(e.table); if (!t) return 0;
    const matched = e.matched.filter(id => t.rows.has(id));
    const scanDur = this.scan(t, e, ofs, { yes: 'danger', no: 'dim' });
    if (!matched.length) { this.sched(ofs + scanDur, () => this.note('0 rows matched:
nothing delet…
    const target = this.binTarget(t, matched), bin = this.spawn(makeBin(matched.length >
3 ? 1.1 : …
    bin.group.position.copy(this.offstageLeft(target.z + .6));
    this.walk(bin, this.offstageLeft(target.z + .6), target, ofs + 100, 1000, Math.PI
* .38);
    const t0 = ofs + Math.max(1150, scanDur + 150); let at = t0;
    this.tl(t0 - 150, 260, p => { bin.lid = p; });
    const mouth = () => target.clone().add(V(0, bin.mouthY + .15, 0));
    matched.forEach((rid, k) => {
      const when = t0 + 200 + k * 360;
      this.sched(when, () => {
```

Sample 4: a challenge is plain data

```
  'Removing Rows with DELETE': [
    diy('Delete the pet called Bo.', 'name\nRex\nTom\nAnn',
      "DELETE FROM pets\nWHERE name = 'Bo';",
      "DELETE FROM pets WHERE name = 'Bo';",
      [{ label: 'Uses DELETE FROM pets', re: 'delete\\s+from\\s+pets' }], 'DELETE removes
the rows …
```

Coupling, cohesion, simplicity, modularity and readability

Table 5.3: How the code shows the required characteristics

Characteristic	Evidence in LADO
Low coupling	Content files know nothing about the scenes. Engines return plain frame data; worlds only read frames. A scene can be changed without touching an engine.
High cohesion	Each module does one job: sql-model.js only parses and runs SQL; sql-kit.js only builds meshes; sql-scene.js only replays frames and draws the screen.
Simplicity	Lessons and challenges are data objects. The engines use a small recursive-descent parser. Each command has one animation.
Modularity	One module per environment, per engine, per world. ES modules import only what they need.
Readability	Short named functions (onDelete, onInsert, scan), comments that state purpose, and a consistent style across files.

Quality of the program

Table 5.4: Quality features of the program

Quality	How it is provided
User friendliness	Simple English notes, one task at a time, a live checklist, praise on success, large buttons.
Navigation	Topic cards, side menu (slide-in on phones), Previous and Next buttons, direct links such as index.html#sql/sql-topic-1/4.
Help facilities	Hint button, answer after a second press, steps under each example, plain-English error hints.
Colour combination	Dark and light themes; syntax colours; green for correct, red for errors, yellow for attention.
Appropriate terminology	Everyday words first (a table is a grid, a bin takes rows away); technical term shown in bold beside it.
Error handling	Every engine error becomes a readable message with a hint; incomplete code while typing shows “Keep typing” instead of an error.

Program resilience

- Security: code runs only inside interpreters in the browser and cannot reach the file system or the network.
- Initialisation: each lesson sets its own starting tables or code, so tasks never depend on earlier tasks.

- Termination: endless loops, endless recursion and very long runs are stopped by limits and reported in simple words; scenes are disposed when a lesson is closed so that graphics memory is released.

The interface in use

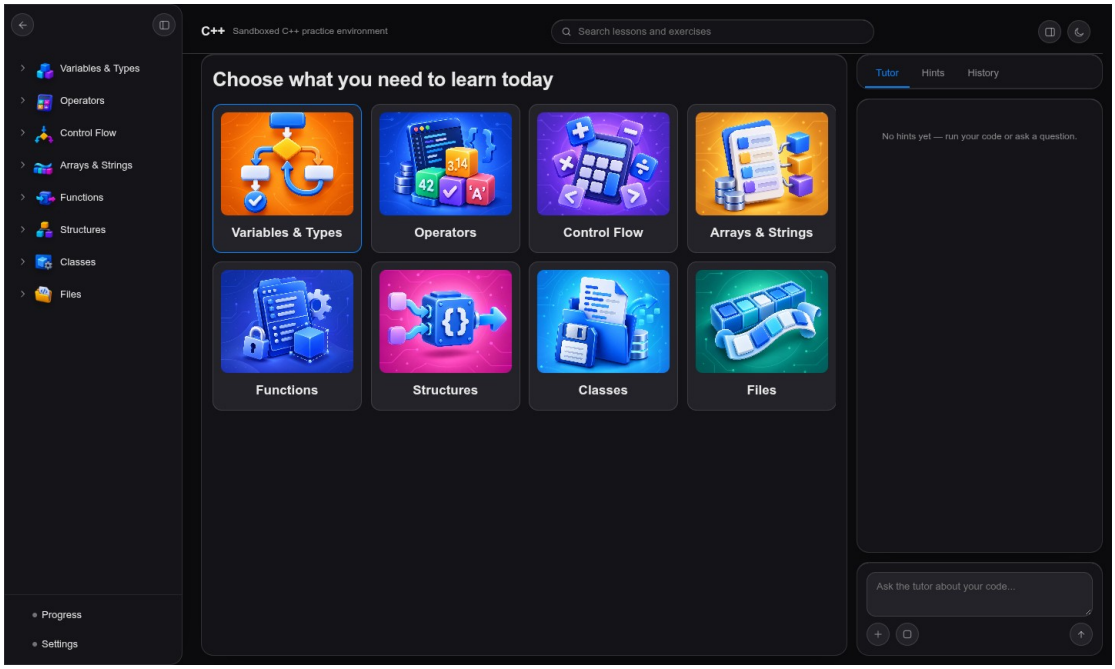


Figure 5.1: Topic cards of the C++ environment

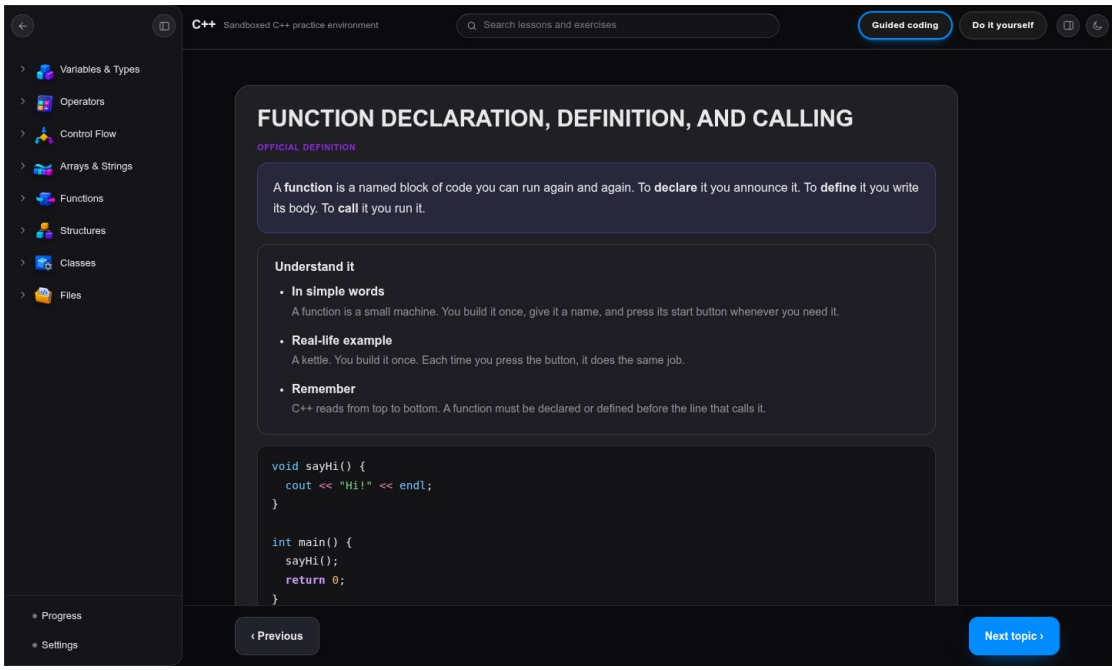


Figure 5.2: Notes card for a C++ subtopic

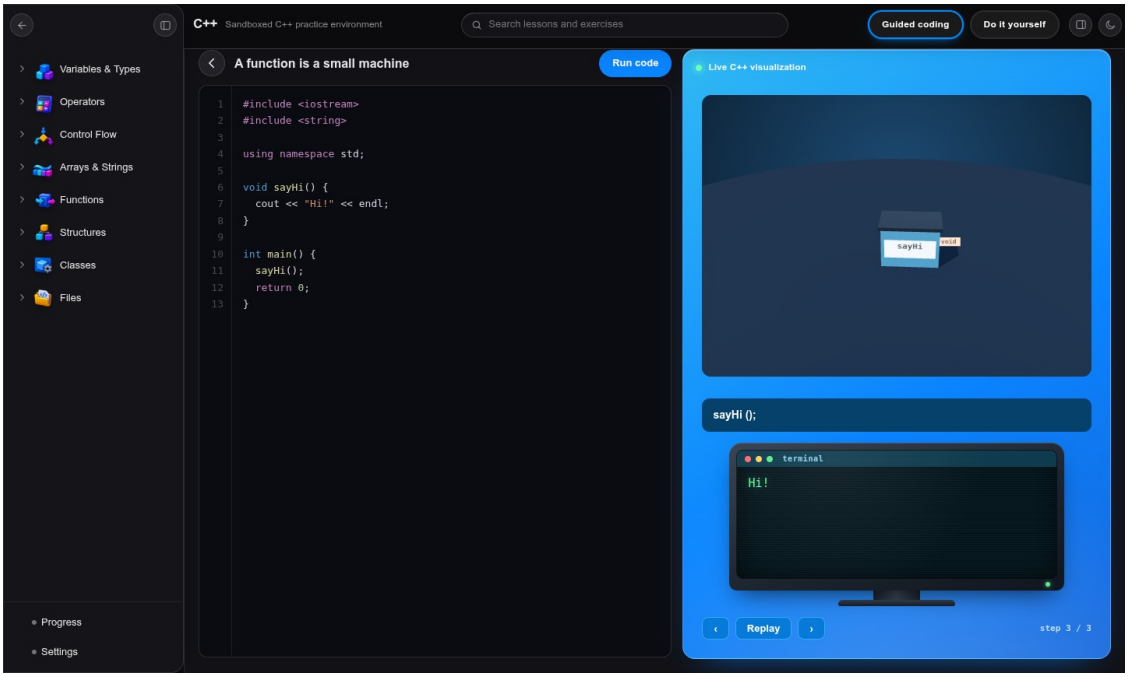


Figure 5.3: Guided typing of a function with the live 3D scene and output computer

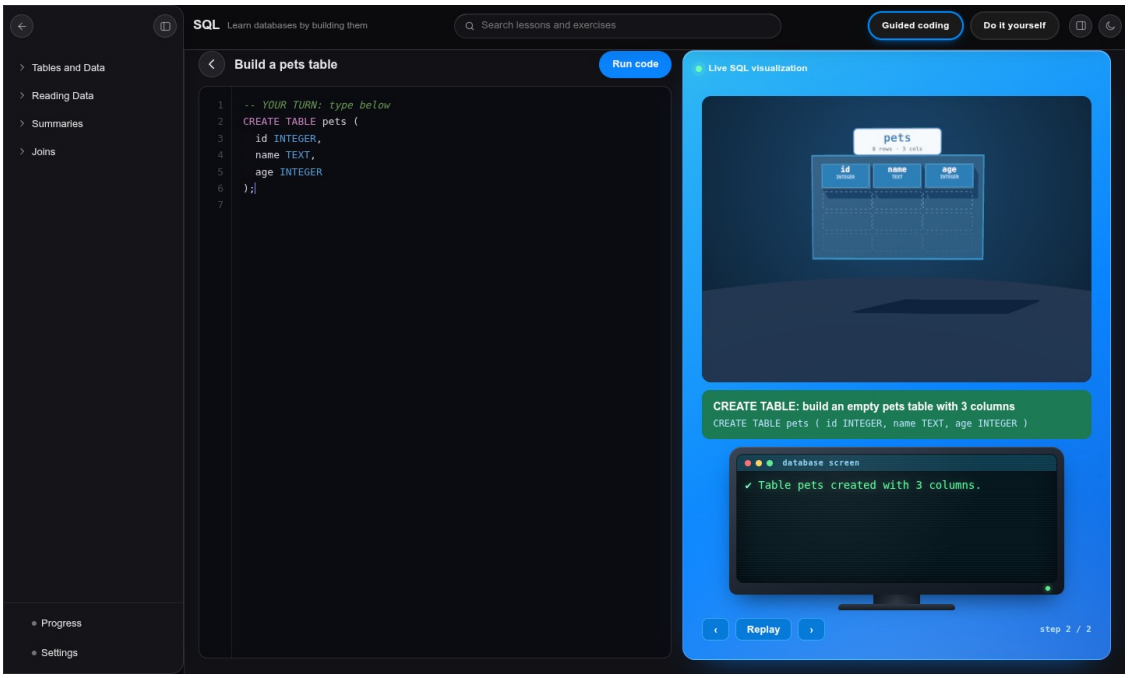


Figure 5.4: SQL: CREATE TABLE builds the table column by column

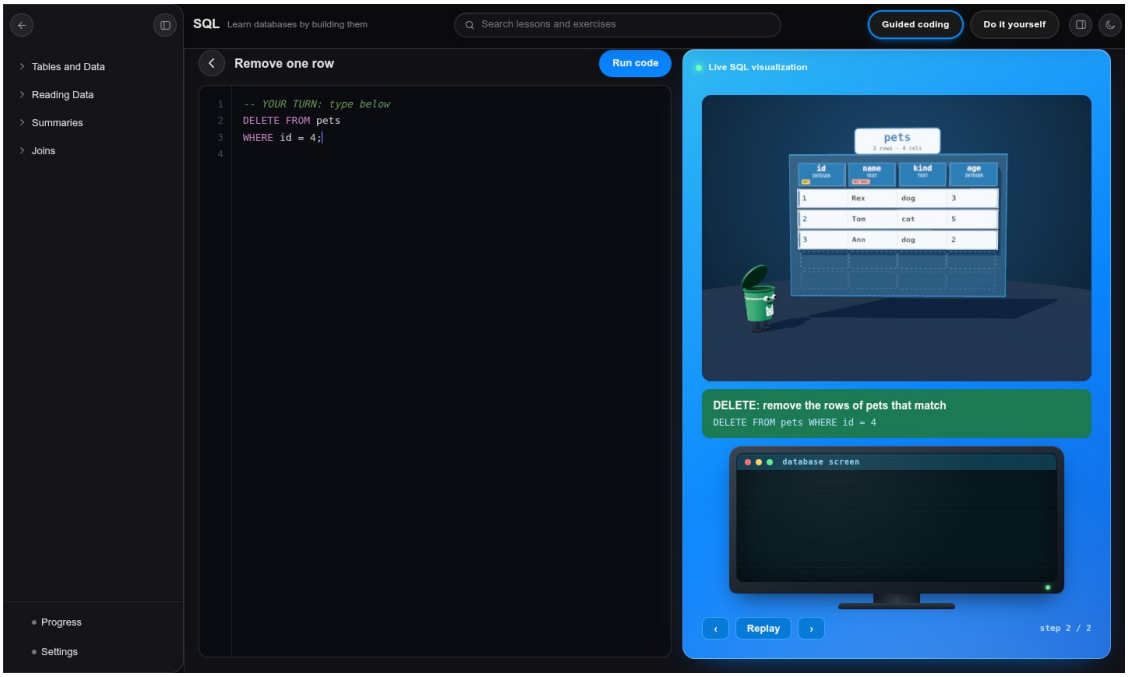


Figure 5.5: SQL: DELETE sends a bin to the matching rows

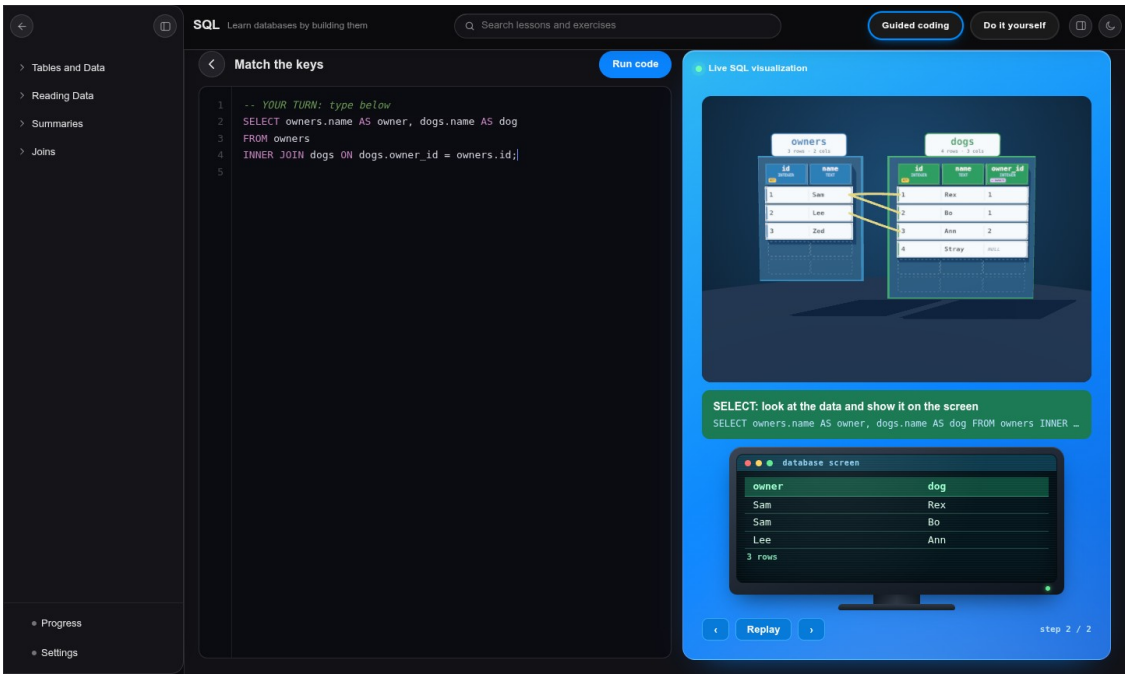


Figure 5.6: SQL: a join links matching rows and the result appears on the computer

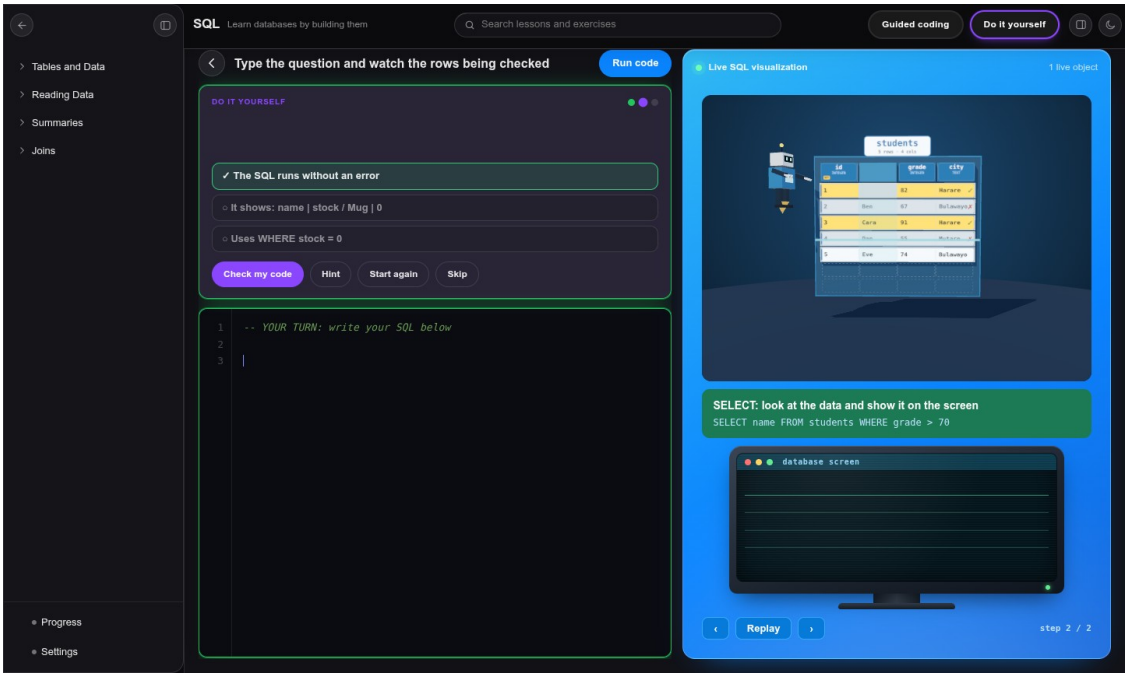


Figure 5.7: A Do it yourself task with the live goal checklist

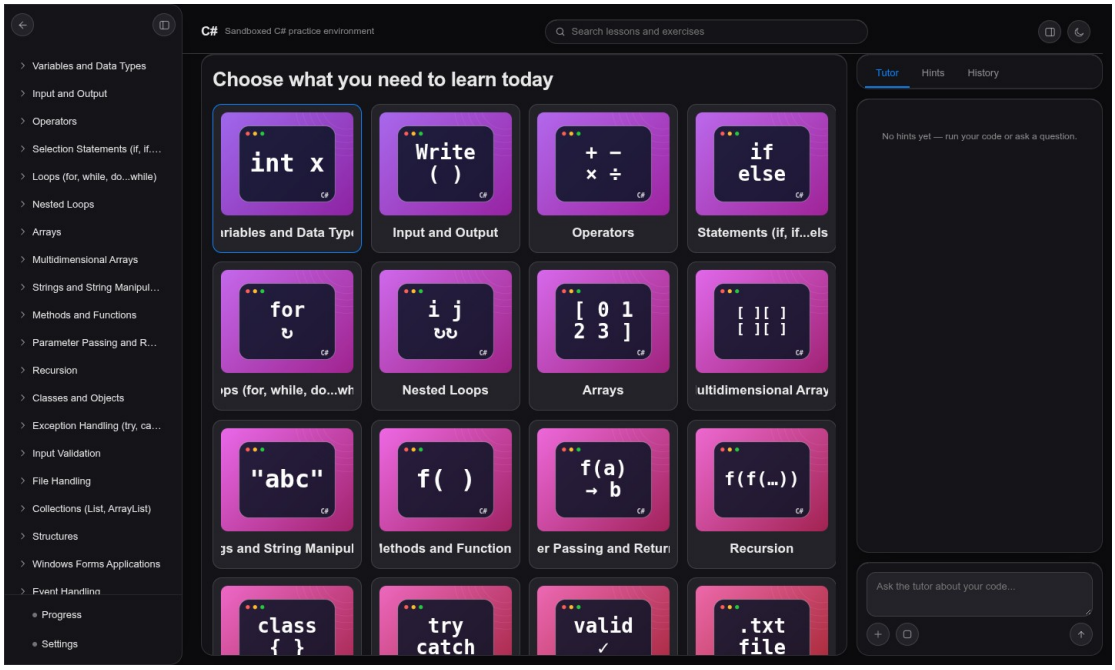


Figure 5.8: Topic cards of the C# environment

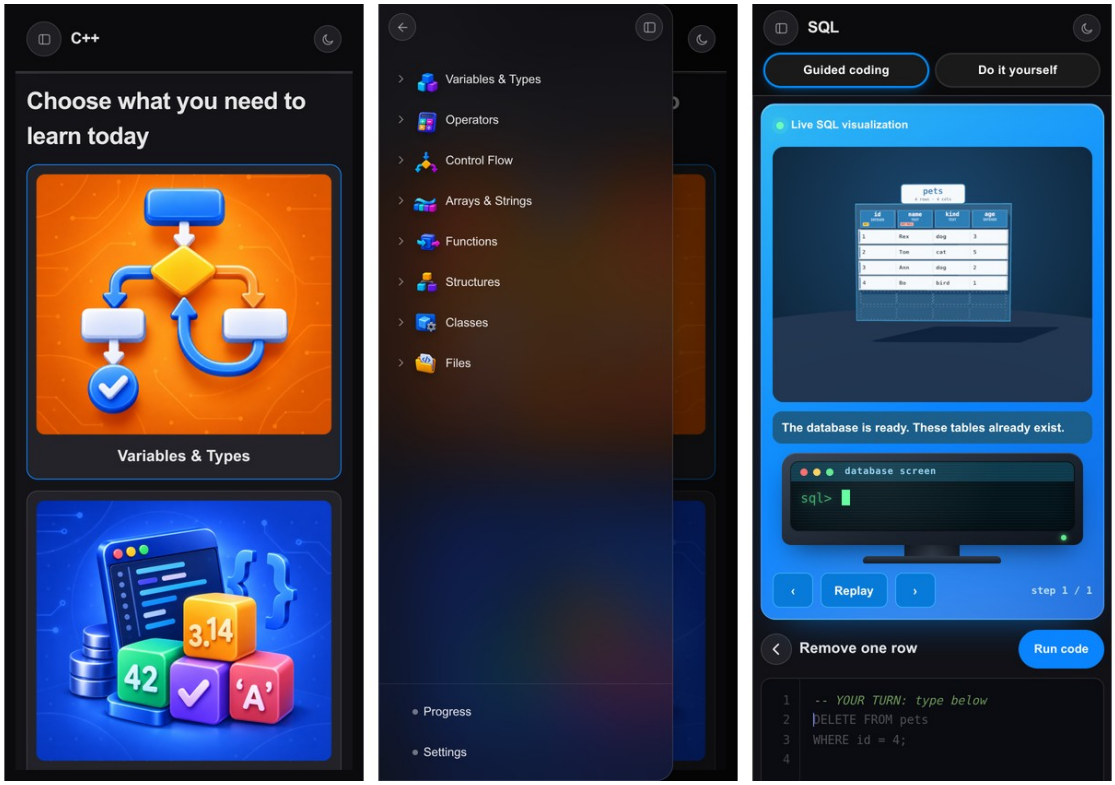


Figure 5.9: Phone layout: topics, lesson menu and coding sheet

5.4 Testing

The test plan used four kinds of data: normal data (typical input that should work), invalid data (wrong input that should give a helpful error, not a crash), boundary data (values at the edge, such as an empty table or the last array index) and peak data (large or endless input). The tests were run on 14 October 2026 with Node.js 20 against the real engines of the project, so that the results below are measured, not assumed. Time is the time taken in milliseconds on the developer laptop.

Table 5.5: Test results: SQL engine

ID	Data	Test	Expected	Actual	Result	ms
S1	Normal	Select all rows	4 rows	4 rows	Pass	3.8
S2	Normal	Filter with WHERE and ORDER BY	Ann, Rex, Tom	Ann, Rex, Tom	Pass	1.3
S3	Normal	Aggregate with GROUP BY	dog 2, cat 1, bird 1	dog 2, cat 1, bird 1	Pass	1
S4	Normal	INSERT then SELECT	5	5	Pass	1.1
S5	Normal	UPDATE with WHERE	Rex 4, Ann 3	Rex 4, Ann 3	Pass	1.3
S6	Normal	DELETE with WHERE	2 rows left	2 rows left	Pass	0.6
S7	Normal	INNER JOIN	3 rows	3 rows	Pass	0.6

ID	Data	Test	Expected	Actual	Result	ms
S8	Normal	LEFT JOIN keeps unmatched owner	4 rows incl. Zed NULL	4 rows, Zed -> null	Pass	0.4
S9	Invalid	Misspelled command	syntax error with a hint	near "SELCT": syntax error | Did you mean SELECT?	Pass	0.5
S10	Invalid	Unknown table	no such table	no such table: pet	Pass	0.3
S11	Invalid	Unquoted text value	no such column with quote hint	no such column: dog	Pass	0.3
S12	Invalid	NOT NULL rule broken	NOT NULL constraint failed	NOT NULL constraint failed: pets.name	Pass	0.2
S13	Invalid	Duplicate primary key	UNIQUE constraint failed	UNIQUE constraint failed: pets.id	Pass	0.2
S14	Invalid	Text in a number column	datatype mismatch	datatype mismatch: pets.age holds whole numbers, but got 'old'	Pass	0.3
S15	Invalid	Aggregate inside WHERE	misuse of aggregate	misuse of aggregate function COUNT()	Pass	0.5
S16	Invalid	Unfinished statement while typing	incomplete flag, no crash	incomplete=true	Pass	0.2
S17	Boundary	LIMIT 0	0 rows	0 rows	Pass	0.3
S18	Boundary	SELECT on an empty table	0 rows, no error	0 rows, error=none	Pass	0.3
S19	Boundary	COUNT on empty set	0	0	Pass	0.2
S20	Boundary	AVG on empty set is NULL	NULL	null	Pass	0.2
S21	Boundary	BETWEEN includes both ends	Rex, Ann, Bo	Rex, Ann, Bo	Pass	0.4
S22	Peak	Insert 5,000 rows then count	5005 rows	5004 rows	Pass	3276.4
S23	Peak	Group 5,000 rows	15 groups	15 groups	Pass	3625.4

Table 5.6: Test results: C++ teaching interpreter

ID	Data	Test	Expected	Actual	Result	ms
C1	Normal	Print a sum	5	5	Pass	5.3
C2	Normal	for loop	1 2 3	1 2 3	Pass	1.9
C3	Normal	if / else	Teenager	Teenager	Pass	0.2
C4	Normal	Class with constructor	7	7	Pass	1.4
C5	Normal	Recursion factorial(5)	120	120	Pass	0.7
C6	Invalid	Missing semicolon	error asking for ;	Expected ";", but found "cout".	Pass	0.1

ID	Data	Test	Expected	Actual	Result	ms
C7	Invalid	Undeclared variable	not declared error	x has not been declared yet.	Pass	0.1
C8	Invalid	Unmatched brace	error	(none)	Accepted (by design)	0.1
C9	Boundary	Array last valid index	allowed	3	Pass	0.2
C10	Boundary	Array index equal to size	out-of-range error	Index 3 is outside the 3 items (valid: 0 to 2).	Pass	0.1
C11	Boundary	Loop that runs zero times	no output, no error	output="" error=none	Pass	0.3
C12	Peak	Endless loop is stopped	stopped by step limit	stopped	Pass	6.1
C13	Peak	Loop of 50,000 iterations	1249975000 or stopped safely	stopped safely	Pass	4.3
C14	Peak	Recursion without base case	stack overflow message	f keeps calling itself and never stops (stack overflow). A recursive function needs a base	Pass	2.4

Out of 38 automated tests, 37 passed. Test C8 (a missing closing brace) did not give an error. This is by design: the interpreter tolerates an unfinished program so that the scene can still show the part the learner has typed so far, and the error is shown when the program is run in the challenge checker. Peak tests S22 and S23 passed but took about three seconds for 5,000 inserts, because the engine copies a snapshot of the database after every statement for the 3D replay; this is acceptable for lesson-sized data, and the maintenance plan lists a faster snapshot method.

Content and challenge validation

Besides the engine tests, validation scripts run every content item. For the C++ and SQL environments they ran each guided example and each challenge solution through the engine and checked that (a) every solution passes with all goals ticked, (b) every Debug program fails as intended, and (c) no Do it yourself task passes with an empty answer. Results: 141 C++ challenges and 63 SQL challenges validated with no problems. The 216 C++ and 63 SQL guided examples all ran as intended; examples marked as warnings produce the intended error or side effect.

Browser testing

An automated headless Chrome run opened lessons of each environment, typed guided examples, completed Do it yourself and Debug tasks, and checked that no script errors

occurred. A phone-sized viewport of 390 by 844 pixels was used to check that the page did not scroll sideways and that the menu opened and closed. Remaining test work: automated tests for the C# environment and testing on physical phones of different makes.

Acceptance testing

Acceptance testing is the last test before the platform is adopted. It is planned for a class of Information Technology students and their lecturer at Masvingo Polytechnic. Each student is given the same four tasks: open a lesson, complete one guided example, solve one Do it yourself task and fix one Debug task. An observer records the time taken and the points where the student hesitates on the observation score sheet (Appendix D), and each student then fills in the questionnaire (Appendix C). The system is accepted if at least 80 percent of students finish all four tasks without help and rate the system four or five on statements 9 and 10 of the questionnaire.

5.5 Installation

LADO is a set of static files. There is nothing to compile or install on the learner's device.

System manual (for the administrator)

1. Copy the project folder to any web server, or publish the repository with GitHub Pages.
2. For a local test, open a terminal in the project folder and run: python3 -m http.server 8000.
3. Open http://localhost:8000 in a browser. For a direct lesson link use a hash such as http://localhost:8000/#sql/sql-topic-1/4.
4. After changing any file, change the version token (for example 20261015-m-1) everywhere it appears so that browsers load the new files.
5. To add a lesson, add a content file in the environment's topics folder, list it in the environment module, and add its challenges to the practice file. Run the validation scripts before publishing.

User manual

The user manual with screenshots is in Appendix A.

5.6 Maintenance

Table 5.7: Maintenance plan

Type	Activity	Frequency
Corrective	Fix reported errors in engines, scenes or content; add a test for each fixed error	As reported
Adaptive	Test with each new major release of Chrome, Edge, Firefox and Safari	Every term
Perfective	Add topics, challenges and scenes; improve snapshot speed in the SQL engine; add automated tests for C#	Every term
Preventive	Run all content validators; check page weight and graphics memory; update the Three.js library after testing	Every term
Backup	The Git repository on GitHub keeps every version; keep one offline copy of the folder	After every release

5.7 Conclusion

The implementation delivered the three environments with notes, guided typing, 3D scenes, and challenges, and the tests show that the engines behave correctly for normal, invalid, boundary and peak data. The one design trade-off found, snapshot cost in large SQL runs, is recorded for maintenance.

Chapter 6: Conclusion

6.1 Future Plans

- Accounts, a score for each learner, and progress dashboards for learners and lecturers (a small server or a hosted database such as Firebase).
- An AI tutor that explains a learner's own mistake step by step, with the guided engines checking its answers.
- A tool that converts a lecturer's question paper into challenges.
- A Linux terminal environment with simulated commands and a file-tree scene.
- Further SQL topics (subqueries, window functions, data design), and deeper C# scenes for classes, files and databases.
- Automated tests for the C# environment, faster database snapshots and installable offline use (a progressive web app).
- More languages, such as JavaScript and Python, using the same engine and scene pattern.

6.2 Conversion Plan

The organisation can move from the current system to LADO with a gradual (phased) conversion so that no teaching time is lost.

Table 6.1: Conversion plan

Step	Activity	Who
1	Publish the platform on the school server or GitHub Pages and test the link in the laboratory	Technician
2	Train lecturers for one hour: where the content is, how to read the notes, how to add a challenge	Developer
3	Pilot with one class for four weeks while the normal system continues (parallel running)	Lecturer
4	Collect feedback with the questionnaire in Appendix C and fix problems	Developer
5	Roll out to all classes and use LADO for weekly practice homework	Head of department
6	Keep the compiler-based laboratory sessions for real tools; LADO replaces the unsupervised practice	Lecturers

6.3 Limitations

- Progress and scores are not saved, and there are no accounts.
- The engines run a teaching subset of C++, C# and SQL, not the complete languages.
- The Linux environment, the AI tutor and the question-paper converter of the original proposal are not built.
- The C# environment mostly reuses the C++ scenes and has no automated tests yet.
- The 3D scenes need WebGL and run best on a laptop; very old phones may show a simpler view.
- Acceptance testing with a full class at Masvingo Polytechnic is planned (Section 5.4) but has not yet been carried out, so there are no adoption figures yet.

6.4 Recommendations

- Adopt LADO as a free practice tool alongside laboratory sessions and let lecturers write new challenges from their own exercise sheets.
- Start with the SQL and C++ environments, which are the most complete, and build the accounts and dashboards next so that lecturers can see class progress.
- Keep one animation per command and the simple-English style, because they are what make the lessons easy to follow.
- Run the validators every time content changes, and keep the repository as the single source of truth.

Bibliography

The references are in Harvard style and in alphabetical order.

Ben-Ari, M. (2001), Constructivism in computer science education, Journal of Computers in Mathematics and Science Teaching, 20(1), pp. 45-73, Association for the Advancement of Computing in Education, USA.

Deterding, S., Dixon, D., Khaled, R. and Nacke, L. (2011), From game design elements to gamefulness: defining gamification, Proceedings of the 15th International Academic MindTrek Conference, pp. 9-15, ACM, USA.

Guo, P. (2013), Online Python Tutor: embeddable web-based program visualization for CS education, Proceeding of the 44th ACM Technical Symposium on Computer Science Education, pp. 579-584, ACM, USA.

Jakazira, I. (2026), LADO (Learn As you Do): project proposal, National Diploma in Information Technology, Zimbabwe.

Mayer, R. E. (2009), Multimedia Learning, 2nd edn, Cambridge University Press, United Kingdom.

Prince, M. (2004), Does active learning work? A review of the research, Journal of Engineering Education, 93(3), pp. 223-231, American Society for Engineering Education, USA.

Sommerville, I. (2016), Software Engineering, 10th edn, Pearson Education, United Kingdom.

Sorva, J. (2013), Notional machines and introductory programming education, ACM Transactions on Computing Education, 13(2), Article 8, ACM, USA.

Sweller, J. (1988), Cognitive load during problem solving: effects on learning, Cognitive Science, 12(2), pp. 257-285, Cognitive Science Society, USA.

VanLehn, K. (2011), The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems, Educational Psychologist, 46(4), pp. 197-221, Routledge, USA.

three.js (2024), three.js documentation, available at https://threejs.org/docs/ (Accessed 14/10/2026).

MDN Web Docs (2024), WebGL: 2D and 3D graphics for the web, available at https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API (Accessed 14/10/2026).

Appendix A: User Manual

A.1 Opening the platform

Open the link given by your lecturer in a web browser. The home page asks “What do you want to learn?”. Choose C++, C# or SQL with the buttons at the bottom of the page. The moon and sun button changes between the dark and light look.

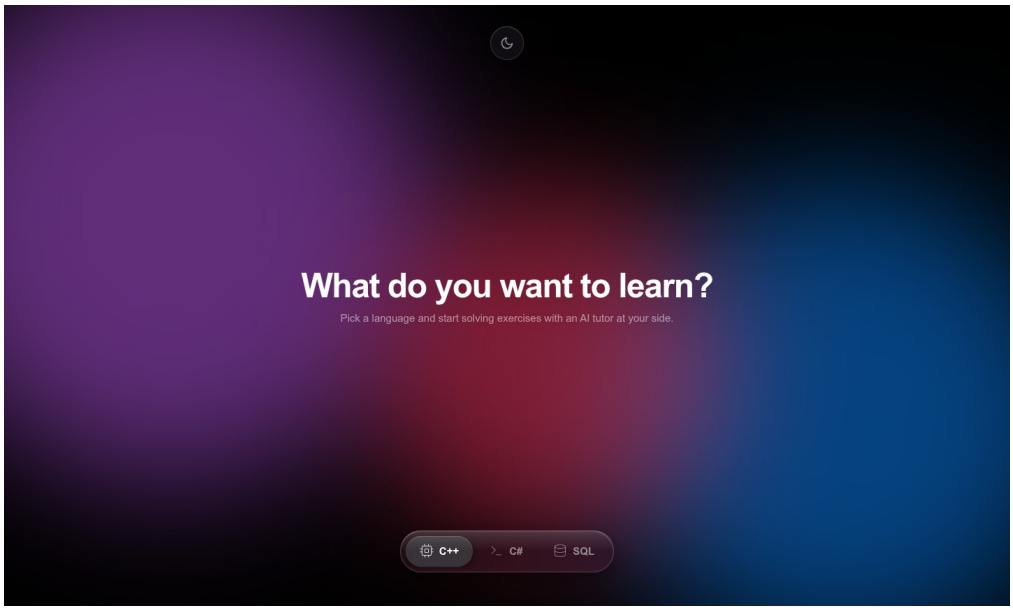


Figure A.1: Home page with the language buttons

A.2 Choosing a lesson

1. Tap a topic card, for example “Tables and Data”.
2. Tap a subtopic card. The notes page opens.
3. On a phone, tap the menu button at the top left to open the list of lessons, and tap a lesson to go to it.
4. Use the Previous and Next buttons at the bottom to move between lessons.

A.3 Reading the notes

Each notes page has a title, an official definition, a card explaining the idea in simple words, a real-life example, a code sample, and then cards for further parts. Under each part there is a button Guided coding.

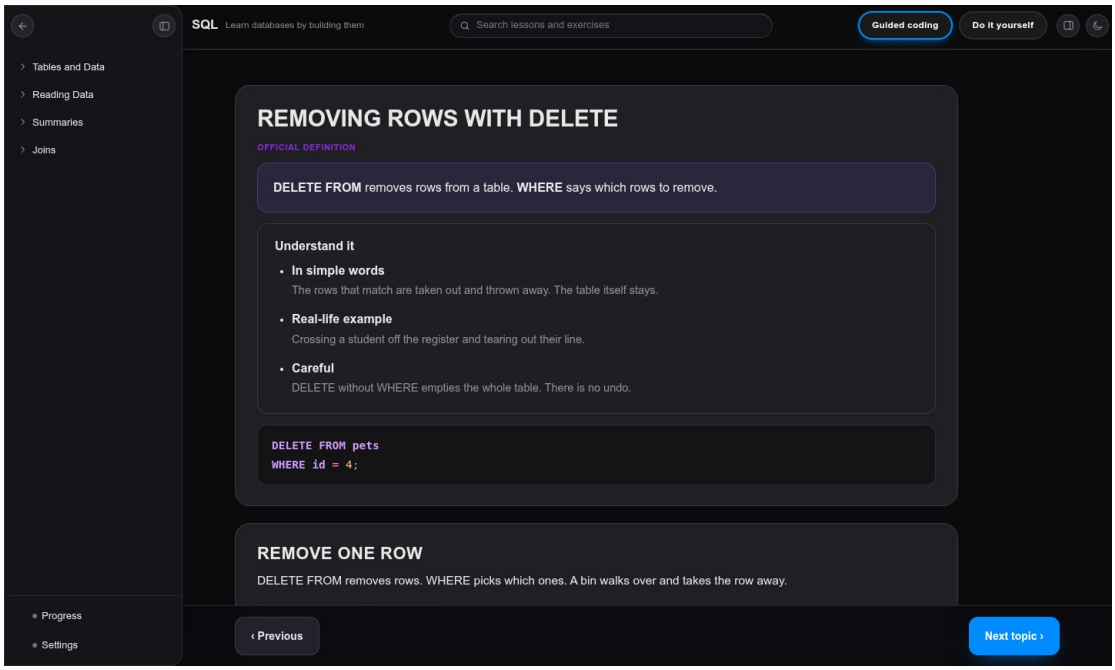


Figure A.2: A notes page

A.4 Guided coding

1. Press Guided coding under an example. The coding sheet opens with the scene and the computer.
2. Click in the editor and type. Grey ghost text shows what to type next. In SQL you may type in small letters; LADO changes them to capitals.
3. If you press a wrong key, the editor shakes and shows the expected text.
4. Watch the scene build as you type. When you stop, the whole program replays step by step and the result appears on the computer screen.
5. Use the buttons under the scene to pause, replay, or step back and forward.

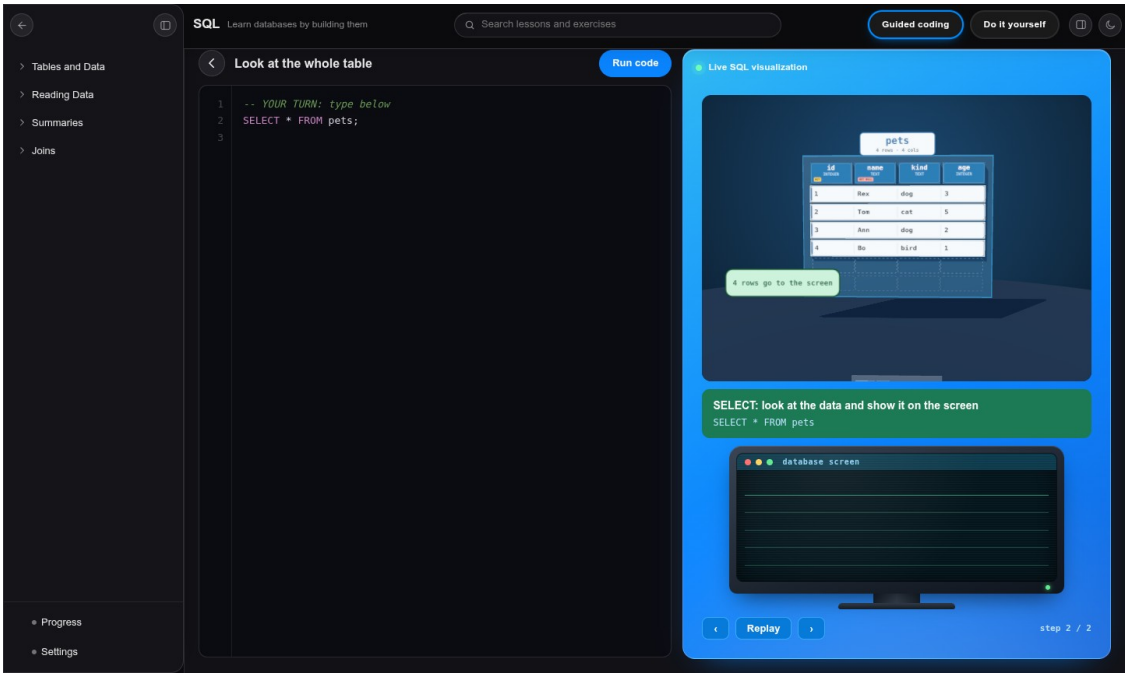


Figure A.3: Guided coding with the robot copying a table to the screen

A.5 Do it yourself and Debug

1. Press Do it yourself at the top of the page. A bar shows the task and its goals.
2. Write your answer in the editor. Each goal gets a tick when it is true.
3. When every goal is ticked, the task is correct and the next task starts. You can also press Check my code.
4. Press Hint for a clue; press it again for the full answer. Press Skip to move on and Reset to start the task again.
5. In a Debug task the code already has a mistake. Read the error, find the bug and fix it until all goals are ticked.

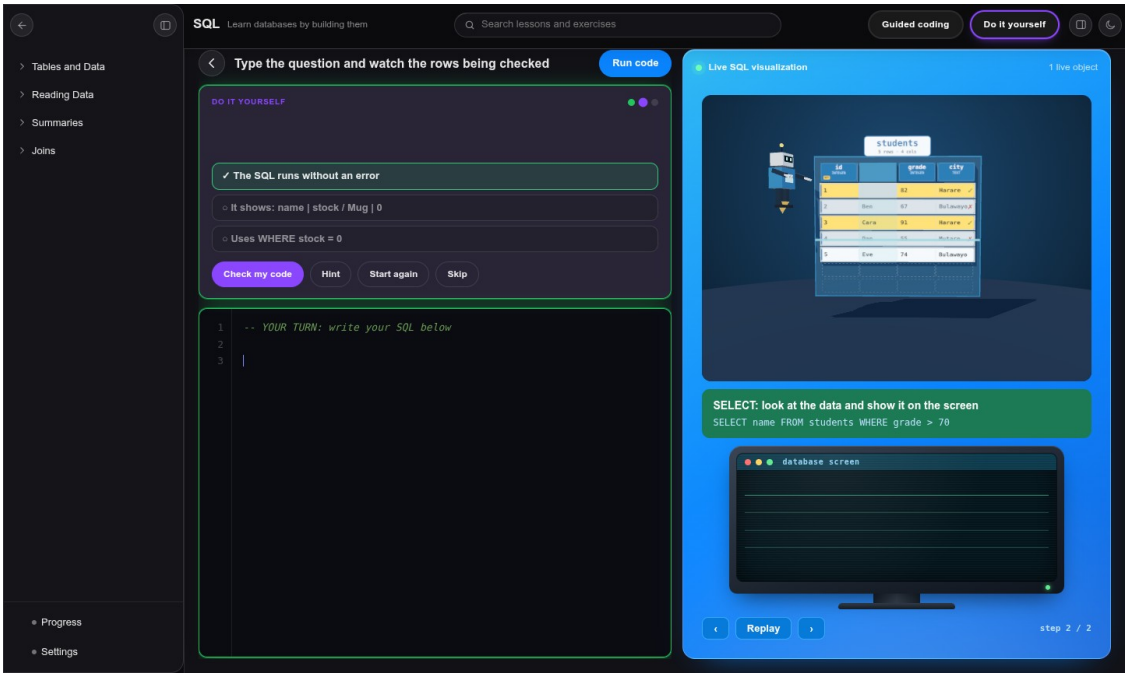


Figure A.4: A task with the goal checklist

A.6 Using a phone

On a phone the page uses one column. The lesson list is a menu. The coding sheet shows the scene on top and the editor below; scroll down to the editor.

A.7 If something does not work

- Hard refresh the page (hold Shift and press reload) to load the newest files.
- If the 3D scene is blank, check that the browser supports WebGL, or try Chrome or Edge.
- If a task is not accepted, read the first goal without a tick: it says what is still missing.

Appendix B: Interview Checklist

Respondent: _____ Role: _____ Date: ____ / ____ / 2026 Interviewer: _____

Table B.1: Interview checklist

No.	Question or topic	Notes taken
1	How are practical programming and database lessons organised each week?	
2	How many hours of laboratory time does each class get? Is it enough?	
3	What software is installed on the laboratory computers? Who maintains it?	
4	What do students find hardest to understand? Which topics?	
5	What happens when a student gets stuck during practice?	
6	How are practical exercises marked and how long does feedback take?	
7	Which devices do students own (phones, laptops)? Do they have internet?	
8	What features would make an online practice tool useful to you?	
9	Who could write new exercises and how would they prefer to do it?	
10	What worries do you have about using a new system?	

B.2 Interview responses register

The interview answers are recorded as one row for each respondent, in the layout of a Google Forms responses sheet. Question numbers in the columns refer to the checklist in Table B.1. Real answers can replace the sample rows by pasting the exported rows into the grids.

Table B.2: Legend of the interview questions

Column	Topic of the question
Q1	Weekly organisation of practical lessons
Q2	Laboratory hours per class and whether they are enough
Q3	Software installed on laboratory computers and who maintains it
Q4	Topics students find hardest
Q5	What happens when a student gets stuck
Q6	How practical work is marked and how long feedback takes
Q7	Devices and internet access of students
Q8	Features wanted in an online practice tool
Q9	Who could write new exercises
Q10	Worries about using a new system

Sample data: the answers in the registers below are illustrative examples prepared for this document to show how collected answers are recorded and analysed. They are not answers collected from real respondents.

Table B.3: Interview responses register, questions 1 to 5 (sample data)

No.	Timestamp	Code	Role	Q1 Organisation	Q2 Lab hours	Q3 Software	Q4 Hardest topics	Q5 When stuck
1	2026-09-02 09:14	R1	Lecturer (programming)	One lecture and one lab per week	Two hours per class, not enough	Compiler on lab PCs, technician maintains	Loops, arrays and SQL joins	Waits for lecturer or asks a friend
2	2026-09-02 11:40	R2	Lecturer (databases)	Theory first, then SQL exercises	Two hours, shared PCs	Database tool, sometimes missing	Joins and GROUP BY	Copies a solution or stops
3	2026-09-03 08:55	R3	Lab technician	Labs follow the timetable	Rooms are full on test weeks	Windows and compilers, updates slow	Installation problems	Students call him for errors
4	2026-09-03 14:20	R4	Head of department	Weekly lecture plan per module	Limited by room and staff numbers	Licensed IDEs, limited budget	Practical coding	Extra lessons when possible
5	2026-09-04 10:05	R5	Student representative	Labs are crowded	Not enough, we queue for PCs	Software often not working	Loops and functions	We guess until it works
6	2026-09-04 12:30	R6	Lecturer (programming)	Mixed theory and practice	Too short to finish tasks	Needs installation of tools	Classes and objects	Leaves the exercise for later

Table B.4: Interview responses register, questions 6 to 10 (sample data)

No.	Code	Q6 Marking and feedback	Q7 Devices	Q8 Wanted features	Q9 Who writes exercises	Q10 Worries
1	R1	Marked in the next lesson	Most have phones, few have laptops	Instant feedback, step-by-step help	Lecturers could add exercises	Will students stop coming to class?
2	R2	Marked after a week	Phones and shared laptops	A SQL practice space with answers	Lecturers with templates	Data must stay private
3	R3	Not involved in marking	Phones, data is expensive	Works without installation	Not applicable	Needs little maintenance
4	R4	Moderated at end of term	Many students use phones	Low cost and works offline	Lecturers, checked by the department	Cost and staff training
5	R5	Feedback comes late	Phone only	Games and animations	Lecturers	Too hard to use
6	R6	Marked in batches	Phones and a few laptops	Shows what the code does	Lecturers and top students	Time to prepare new content

Appendix C: Questionnaire

Student questionnaire. Please tick one box for each statement. Your answers are anonymous. Scale: 1 = strongly disagree, 2 = disagree, 3 = not sure, 4 = agree, 5 = strongly agree.

Table C.1: Student questionnaire

No.	Statement	1	2	3	4	5
1	I get enough practice with programming during lessons.					
2	When I get stuck, I can find help quickly.					
3	Error messages help me to understand my mistake.					
4	I can picture what my program does inside the computer.					
5	I have a device I can use to practise outside the laboratory.					
6	I would use a website that lets me practise with instant feedback.					
7	Animations that show what code does would help me learn.					
8	Short notes in simple English help me more than long notes.					
9	The system was easy to use. (after trying LADO)					
10	The system helped me to understand the topic. (after trying LADO)					

Open questions: 1. What did you like most about the system? 2. What should be changed? 3. Which topics should be added?

C.2 Questionnaire responses register

Each learner who completes the questionnaire is one row. Scores are 1 (strongly disagree) to 5 (strongly agree). The sheet below follows the layout of a Google Forms export, so exported rows can be pasted straight in. The summary table that follows is calculated from the register.

Sample data: the scores and comments below are illustrative examples prepared for this document. They are not answers collected from real learners.

Table C.2: Questionnaire responses register (sample data)

No.	Timestamp	Learner	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Open comment
1	2026-09-08 09:10	L01	2	2	2	2	4	5	5	5	4	5	More practice please
2	2026-09-08 10:13	L02	1	2	3	2	5	5	4	5	5	5	Animations are clear
3	2026-09-08 11:16	L03	2	3	2	1	3	4	5	4	4	4	Needs more topics
4	2026-09-09 12:19	L04	3	2	2	2	5	5	5	5	5	5	Easy on my phone
5	2026-09-09 13:22	L05	2	1	2	2	4	4	5	5	4	4	
6	2026-09-09 09:25	L06	2	2	3	3	4	5	4	4	5	5	The bin is fun
7	2026-09-10 10:28	L07	1	2	1	2	5	5	5	5	5	5	I finally see loops
8	2026-09-10 11:31	L08	3	3	2	2	3	4	4	5	4	4	Add Python too
9	2026-09-10 12:34	L09	2	2	2	1	4	5	5	4	5	5	
10	2026-09-11 13:37	L10	2	3	3	2	5	5	4	5	4	4	Hints help a lot
11	2026-09-11 09:40	L11	1	2	2	2	4	4	5	5	5	5	Good for revision
12	2026-09-11 10:43	L12	3	2	3	3	2	4	4	4	4	3	Needs a laptop screen
13	2026-09-12 11:46	L13	2	2	2	2	5	5	5	5	5	5	
14	2026-09-12 12:49	L14	2	1	3	2	4	5	4	5	4	5	Please add scores
15	2026-09-12 13:52	L15	3	3	2	3	4	4	5	4	4	4	

Sample data: the summary below is calculated from the sample register above (15 sample learners).

Table C.3: Summary of the questionnaire scores (sample data)

Statement	Count of 1	Count of 2	Count of 3	Count of 4	Count of 5	Mean score	Percent agree (4 or 5)
Q1 Enough practice	3	8	4	0	0	2.1	0%
Q2 Help is quick to find	2	9	4	0	0	2.1	0%
Q3 Error messages help	1	9	5	0	0	2.3	0%
Q4 I can picture what my program does	2	10	3	0	0	2.1	0%
Q5 I have a device to practise on	0	1	2	7	5	4.1	80%
Q6 Would use a practice website	0	0	0	6	9	4.6	100%
Q7 Animations would help	0	0	0	6	9	4.6	100%
Q8 Short simple notes help	0	0	0	5	10	4.7	100%
Q9 LADO was easy to use	0	0	0	8	7	4.5	100%
Q10 LADO helped me understand	0	0	1	5	9	4.5	93%

Appendix D: Observation Score Sheet

Observer: _____ Date: ____ / ____ / 2026 Learner code: _____ Lesson: _____

Table D.1: Observation score sheet (score 1 = poor, 5 = excellent)

Item observed	Score	Time (min)	Comments
Finds the topic and opens the lesson without help			
Understands the notes card			
Completes the guided typing example			
Links the scene to the code (can explain what happened)			
Completes the first Do it yourself task			
Uses the hint or answer when needed			
Finds and fixes the Debug task			
Looks frustrated or confused (note where)			
Uses the system on a phone without problems			

Appendix E: Snippets of Code

E.1 Guided typing: accepting the expected key (shared-theme.js)

```
      var want = st.body.substr(m.length, text.length);
      if (want !== text) {
        e.preventDefault();
        if (isSql() && want.length === text.length && want.toLowerCase() ===
text.toLowerCase()) { …
        else flash();
      }
    });
    ta.addEventListener('input', function () { guard(); clamp(); st.seen = ta.value;
render(); adva…
```

E.2 Challenge checker (shared-theme.js)

```
    function evaluate(code) {
      var c = list[S.i], res = run(code), ran = !res.error && !res.stopped, goals = [];
      goals.push({ label: sqlLab() ? 'The SQL runs without an error' : 'The program runs
without an…
      goals.push({ label: (sqlLab() ? 'It shows: ' : 'It prints: ') +
trimOut(c.expect).split('\n')…
      (c.goals || []).forEach(function (g) {
        var ok = g.re ? new RegExp(g.re, sqlLab() ? 'i' : '').test(code) : !new
RegExp(g.no, sqlLab…
        goals.push({ label: g.label, ok: ok });
      });
      return { res: res, goals: goals, ok: goals.every(function (g) { return g.ok; }) };
```

E.3 SQL parser: the SELECT statement (sql-model.js)

```
  select() {
    this.next(); const st = { k: 'select', distinct: false, items: [], from: null, joins:
[], where…
    if (this.acceptKw('distinct')) st.distinct = true; else this.acceptKw('all');
    do {
      if (this.isOp('*')) { this.next(); st.items.push({ star: true }); continue; }
      if ((this.peek().t === 'id' || this.peek().t === 'qid') && this.isOp('.', 1) &&
this.isOp('*'…
      const s0 = this.peek().s; const expr = this.expr(); const text = this.src.slice(s0,
this.toks…
      let alias = null; if (this.acceptKw('as')) alias = this.name('a name'); else if
(this.peek().…
      st.items.push({ expr, alias, text });
    } while (this.acceptOp(','));
    if (this.acceptKw('from')) {
      st.from = this.tableRef();
      while (true) {
        let type = null;
        if (this.isKw('inner') && this.isKw('join', 1)) { this.next(); this.next(); type
= 'inner';…
        else if (this.isKw('left')) { this.next(); this.acceptKw('outer');
this.expectKw('join'); t…
        else if (this.isKw('join')) { this.next(); type = 'inner'; }
        else if (this.isKw('cross') && this.isKw('join', 1)) { this.next(); this.next();
type = 'cr…
```

E.4 SQL engine: running a query (sql-model.js)

```
  runSelect(st, { quiet = false, events = null } = {}) {
    const src = this.sourceFor(st);
    const empty = !src.parts.length;
    const scanned = src.rows.map(c => c.rows[0]?.row?.id).filter(x => x !== undefined);
    // WHERE
```

```
    const kept = st.where ? src.rows.filter(c => truthy(this.evalE(st.where, { rows:
c.rows }))) : …
    // output columns
    const items = [];
    st.items.forEach(it => {
      if (it.star) {
        if (empty) throw new SqlError('SELECT * needs a table', { hint: 'Add FROM
table_name.' });
        src.parts.forEach(p => { if (it.table && p.alias.toLowerCase() !==
it.table.toLowerCase() &…
        if (it.table && !items.length) throw new SqlError(`no such table: ${it.table}`);
      } else {
```

E.5 Scene: the timeline runner (sql-world.js)

```
  runTimeline(now) {
    const list = this.timeline; this.timeline = []; const keep = [];
    for (const t of list) {
      if (now < t.a) { keep.push(t); continue; }
      const p = t.b > t.a ? clamp((now - t.a) / (t.b - t.a)) : 1; t.fn?.(p, now);
      if (p >= 1) { t.fn?.(1); t.done?.(); } else keep.push(t);
    }
    this.timeline = [...keep, ...this.timeline];
  }
```

E.6 Scene: table and rows built from data (sql-kit.js)

```
  addRow(rid, vals, { instant = true } = {}) {
    const r = new RowMesh(this, rid, vals); this.rows.set(rid, r); this.order.push(rid);
    r.tz = this.slotZ(this.order.length - 1); r.z = r.tz; r.group.position.set(0, .3
+ .08, r.z); t…
    this.layoutSlots(); this.drawSign(); this.world.layout(); return r;
  }
  removeRow(rid) {
```

Appendix F: Lesson Content Index

Table F.1: Content of the three environments

Environment	Topics	Subtopics	Guided examples	Challenges
C++ environment	Variables & Types (6 subtopics), Operators (8), Control Flow (7), Arrays & Strings (7), Functions (9), Structures (7), Classes (9), Files (8)	61	216	141
C# environment	24 topics from Variables and Data Types to Static Members (one subtopic each)	24	50	33
SQL environment	Tables and Data (6), Reading Data (8), Summaries (4), Joins (3)	21	63	63

Table F.2: Topics of the SQL environment and their scenes

Topic	Subtopics	Scene behaviour
Tables and Data	CREATE TABLE, constraints, INSERT, UPDATE, DELETE, ALTER and DROP	Table assembled column by column; row dropped in and filled; cells flip; bin takes rows; table dropped
Reading Data	SELECT *, columns, WHERE, AND/OR/NOT, LIKE/IN/BETWEEN, ORDER BY, LIMIT/OFFSET/DISTINCT, expressions	Robot scans rows, copies tiles to a tray, sorts and trims them, ships the tray to the computer
Summaries	COUNT, SUM/AVG/MIN/MAX, GROUP BY, HAVING	Rows poured into hoppers or buckets that give one answer each
Joins	Keys, INNER JOIN, LEFT JOIN	Lines link matching rows; unmatched rows are marked NULL